Internalize External Competence for Visual Instruction Editing

Wenjun Huang[*]
Sun Yat-sen University
huangwj98@mail2.sysu.edu.cn

Rui Zhao[*]
National University of Singapore

Suyang Hou
Central South University

Jiahao Tang
Central South University

Wenjia Wang
Sun Yat-sen University

Zhuobai Dong
Wuhan University

Alex Jinpeng Wang[†]
Central South University
jinpengwang@csu.edu.cn

Hu Jian Guo[†]
Sun Yat-sen University
Shenzhen Research Institute
of Sun Yat-Sen University
hujguo@mail.sysu.edu.cn

Abstract

What if you could edit an image simply by drawing on it—circling an object, writing a short label, or sketching an arrow—with minimal or even no typed prompt? We first reveal a surprising capability: when a text-driven image editor is paired with a strong Vision-Language Model (VLM) at inference time, the combined system can directly interpret visual instructions embedded in the image itself, such as on-image text, bounding boxes, and directional arrows. However, this pipeline incurs additional VLM-planning latency, introduces brittle cross-model error propagation, and depends on external APIs or auxiliary model weights. To address these limitations, we introduce Siphon, a framework that internalizes this externally elicited visual-instruction-following competence into a single diffusion-based editor. Rather than relying on an external VLM at inference time, Siphon first uses a VLM planner to synthesize paired visual-instruction supervision, and then transfers this competence into the editor through lightweight LoRA fine-tuning. The resulting model reads, grounds, and executes on-image annotations directly from pixels, while keeping the diffusion sampling cost close to that of the base editor. Across multiple diffusion architectures, Siphon substantially improves spatial controllability, instruction adherence, and marker removal over text-driven baselines, while matching VLM-assisted pipelines without their additional planning latency, API dependence, and cross-model fragility. We do not position rendered long text as a replacement for conventional text prompts; rather, on-image text is mainly intended for short labels and spatially grounded edit intents, while long or complex instructions can still be provided through the standard text channel.

1 Introduction

Imagine editing a photo the way you would mark up a printed document: circling an object you want removed, drawing an arrow toward something you want changed, or simply writing an instruction directly onto the image. This intuitive, visual mode of communication is second nature to humans,

[*]Equal contribution (co-first authors). [†]Co-corresponding authors.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

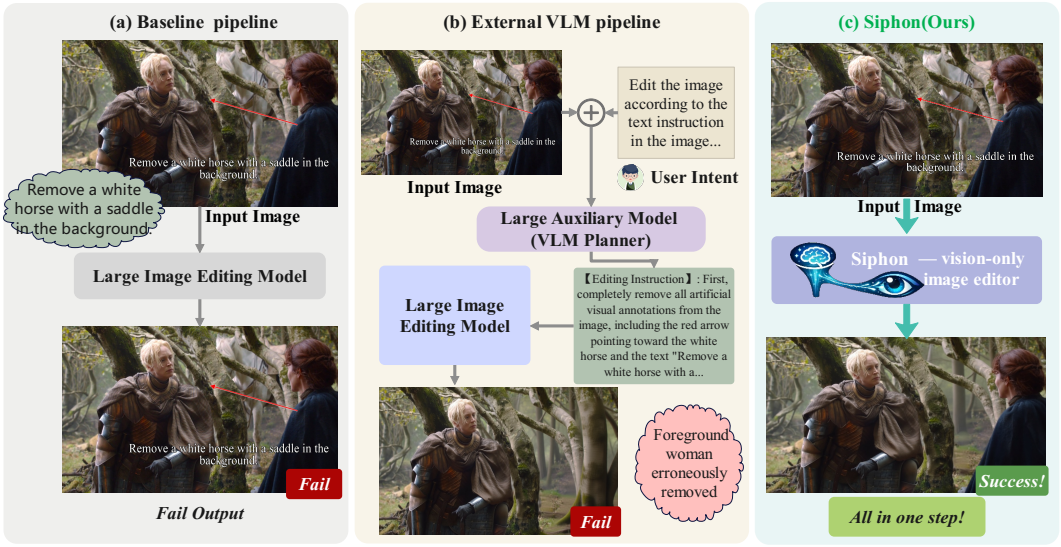


Figure 1: **From External Assistance to Internalized Capability.** *Conventional Pipeline* relies on external VLMs to translate visual intents into text prompts during inference, leading to high latency and potential error propagation. *Our Siphon* internalizes the visual instruction-following capability directly into the editing model, enabling self-contained, end-to-end, vision-only editing without an external VLM planner.

yet remains challenging for text-driven image editing systems that primarily rely on separate textual instructions.

Text-driven image editing has seen remarkable progress in recent years, with diffusion-based models [13, 41] enabling increasingly sophisticated manipulations through natural language descriptions [40, 37, 3]. In practice, product-grade systems often augment these editors with external Large Language Models (LLMs) or Vision-Language Models (VLMs) [1, 51, 21, 50] at inference time to better interpret user intent through prompt rewriting. While effective, such integrations remain shallow, leaving the underlying editor unchanged and still constrained to text-only control at its core [4, 58]. This is a fundamental limitation: natural language is an inefficient medium for precise spatial communication, frequently introducing referential ambiguity.

In this work, we first uncover a powerful yet underexplored capability lurking within existing pipelines. While a standalone text-driven editor may struggle to interpret visual markings, when cascaded with a strong VLM acting as a spatial planner, the combined system can indeed follow visual instructions embedded directly in the input image, including on-image text, bounding boxes, and directional arrows [16, 22, 8, 7], without requiring any manually written textual prompt. This transforms the image itself into a self-contained instruction interface, offering a far more expressive and spatially precise alternative to text-only editing.

Despite its promise, this pipeline approach carries serious practical costs. The sequential execution of an external VLM planner and a diffusion editor introduces an additional pre-processing stage before generation begins. In our latest measurement on a single NVIDIA A100 GPU, the VLM-planning stage takes approximately 30 seconds per image, which is added on top of the already expensive diffusion sampling process. Beyond latency, the system suffers from cross-model fragility, where small failures in the VLM's visual grounding or prompt rewriting can propagate and amplify downstream, producing failed or corrupted edits, as illustrated in Fig. 1(b). Finally, the hard dependency on external APIs or redundant model weights complicates deployment and prevents the editor from being self-sufficient.

To address these limitations, we propose Siphon, a unified framework that internalizes this externally elicited visual instruction-following competence into a single image editing model, as shown in Fig. 1(c). Rather than relying on auxiliary models at inference time, Siphon teaches the editor to directly read, ground, and execute visual instructions from pixels alone. It accomplishes this through two stages. In the elicitation stage, a strong VLM is used as an automated spatial planner to synthesize large-scale paired supervision, composing diverse visual instructions onto images and generating

corresponding clean editing targets. In the internalization stage, this competence is transferred into the diffusion editor via parameter-efficient LoRA fine-tuning [14, 57, 17], enabling fully end-to-end visual instruction editing with the same diffusion sampling cost as the base editor, while removing the extra inference-time VLM-planning step.

We validate Siphon across multiple state-of-the-art diffusion transformer architectures, demonstrating that it is highly generalizable and architecture-agnostic. Extensive experiments show that Siphon substantially outperforms strong text-driven baselines in spatial controllability, instruction adherence, and marker removal, while matching VLM-assisted pipelines without their additional planning latency, deployment burden, and cross-model error propagation. Notably, Siphon enables fully end-to-end vision-only interaction, where users can specify their intent through on-image annotations without an additional manually written prompt [12, 24]. We further use rendered text-to-image examples as a Text-as-Image Forcing curriculum to teach the model to map visual glyphs to semantic concepts. Importantly, we do not argue that long textual instructions should be rendered inside images; on-image text is mainly intended for short labels and spatially grounded edit intents, while long or complex instructions can still be provided through the standard text channel.

To facilitate reproducibility and further research, we will release the training code, model checkpoints, dataset, and the data synthesis pipeline used in this work. Our contributions are summarized as follows:

- We identify and systematically study a previously underexplored capability: when paired with a strong VLM, text-driven image editors can directly follow visual instructions embedded in images, enabling vision-only image editing without any textual prompt.
- We propose Siphon, a framework that internalizes this competence through Text-as-Image Forcing and parameter-efficient adaptation, enabling a single editor to read, ground, execute, and remove visual instructions without an external VLM at inference time.
- Extensive experiments validate the architecture-agnostic generalizability and superior performance of Siphon, demonstrating that it unifies image editing and synthesis within a single end-to-end vision-only paradigm.

2 Related Work

Text-Guided Image Editing. Early works like Prompt-to-Prompt [12] and SDEdit [29] manipulated cross-attention maps or noise levels to preserve structure while altering content. Null-text inversion [32] and Zero-shot I2I [35] further refined these techniques for real image editing. InstructPix2Pix [4], MagicBrush [55], and ImgEdit [53] introduced instruction-tuning datasets, enabling models to follow natural language editing commands. More recently, DiT-based architectures [36] (e.g., Qwen-Image-Edit, SD3 [10]) have scaled this capability to higher resolutions and complex compositions, building on foundational diffusion models [40, 3]. Despite their success, these models rely on a multi-modal input paradigm with separate text encoders (e.g., T5, CLIP) for instruction processing, which often introduces spatial referencing ambiguity. A core challenge of embedding instructions directly into the visual modality is enabling the visual encoder to perceive rendered text as semantic instructions rather than visual textures. While existing typographic generation works like AnyText [43] and TextDiffuser [6] focus on generating coherent text *within* images, we flip this objective: we task the model with generating photorealistic scenes *from* pure text-rendered images, establishing a robust mapping from visual text glyphs to semantic concepts.

Visual Prompting and Spatial Control. Visual prompting seeks to steer model behavior using visual cues rather than language. ViP-LLaVA [5] studies visual prompts for region understanding; Siphon instead learns to execute the indicated edits and remove the annotations. In discriminative tasks, models utilize visual examples for in-context segmentation [39, 44]. In generative contexts, ControlNet [56] and T2I-Adapter [33] introduce side networks to condition diffusion models on spatial maps (e.g., edges, depth, pose). Similarly, IP-Adapter [52] and ControlVAR [18] demonstrate powerful conditioning capabilities using image prompts. However, these methods typically require layout information as separate control tensors, distinct from the RGB input. In contrast, our approach adopts an "In-Context" visual instruction mechanism, where arrows, boxes, and text commands are painted directly onto the input image. We utilize typographic alignment as a pre-training objective to internalize the semantic meaning of visual text, a critical precursor for the model to understand

complex on-image visual instructions. This aligns with recent trends in Multimodal LLMs (e.g., GPT-4o [34], Qwen2.5-VL [1]) that perceive visual markings as semantic instructions, promoting a more intuitive "screenshot-and-edit" interaction paradigm [58, 24].

Unified Multimodal Generation. Recent research has moved towards unifying understanding and generation within single frameworks. Models like OmniGen [48, 47] and UniWorld [20, 19] explore autoregressive or diffusion-based architectures that handle mixed-modal inputs and outputs. Show-o [49] and JanusFlow [26] further integrate these capabilities into unified transformers. Our work complements this direction by specifically focusing on internalizing the editing capabilities elicited from these powerful unified models into efficient diffusion backbones [11].

Synthetic Data Pipelines and VLM Distillation. The scarcity of high-quality editing triplets (source, instruction, target) has driven widespread reliance on synthetic data. InstructPix2Pix [4] first utilized GPT-3 and Stable Diffusion to generate editing supervision. More recent works leverage advanced VLMs for data curation [50, 8, 25], often using VLM-in-the-loop pipelines to build structured supervision for visual understanding and generation. OmniDrive [31] uses LLM-guided spatial planning for multi-view driving video generation. We advance this methodology by establishing a "Generator-Executor" framework. Unlike prior works that use VLMs primarily for captioning or data filtering [2, 25], we employ Qwen-VL-Max for *spatial planning* and *instruction synthesis*. By pairing a VLM planner with a DiT executor, we autonomously generate a large-scale dataset of complex spatial visual instructions, effectively distilling the VLM's spatial reasoning capabilities into the image editor.

3 Method

Our goal is to learn a vision-only image editing model that maps a visually annotated image to a clean edited result, without relying on textual prompts or external reasoning models at inference time. We aim to learn a mapping

$$f_\theta : I_{\text{marked}} \rightarrow I_{\text{edited}}, \tag{1}$$

where I_{marked} contains visual instructions (e.g., text overlays and visual markers) embedded in the image, and I_{edited} is the desired output in which the specified edit is executed and all instruction artifacts are removed.

To this end, we introduce the Siphon framework, as shown in Fig. 3, which consists of two stages. In the elicitation stage, we use a strong vision-language model (VLM) to externalize visual instruction-following competence and automatically synthesize paired supervision. In the internalization stage, we distill this externally elicited competence into a single diffusion-based image editor via parameter-efficient fine-tuning, enabling end-to-end visual instruction editing.

3.1 Eliciting Visual Instruction Following

A central challenge in visual instruction editing is the lack of paired training data: while visual markers are intuitive for humans, existing models do not natively understand them, making it prohibitively expensive to manually construct ground-truth (marked image, edited image) pairs. We address this challenge by introducing a scalable multimodal data engine that leverages a VLM as an external planner to synthesize high-quality supervision.

We categorize visual instructions into four types based on how intent is conveyed, as illustrated in Fig. 2. Type I renders a textual description on a blank canvas and requires the model to generate a realistic image solely from rendered text. Type II embeds textual commands directly onto natural images to specify global edits. Type III uses directional arrows accompanied by text to indicate object-specific modifications. Type IV employs bounding boxes with labels to specify region-constrained insertions. Together, these instruction types span image generation, global semantic edits, and spatially grounded local operations. Practical use cases for each instruction type, including why visual-text instructions are useful beyond synthetic rendering, are discussed in Appendix C.2.

For instruction types involving complex visual markers, Types II–IV, directly generating a clean target image is non-trivial. We therefore adopt a VLM-in-the-loop strategy, using a Large Auxiliary Model (specifically, Qwen-VL-Max in our implementation) as a visual planner. Given a marked image I_{marked}, the VLM first performs scene analysis and visual grounding to interpret the user intent

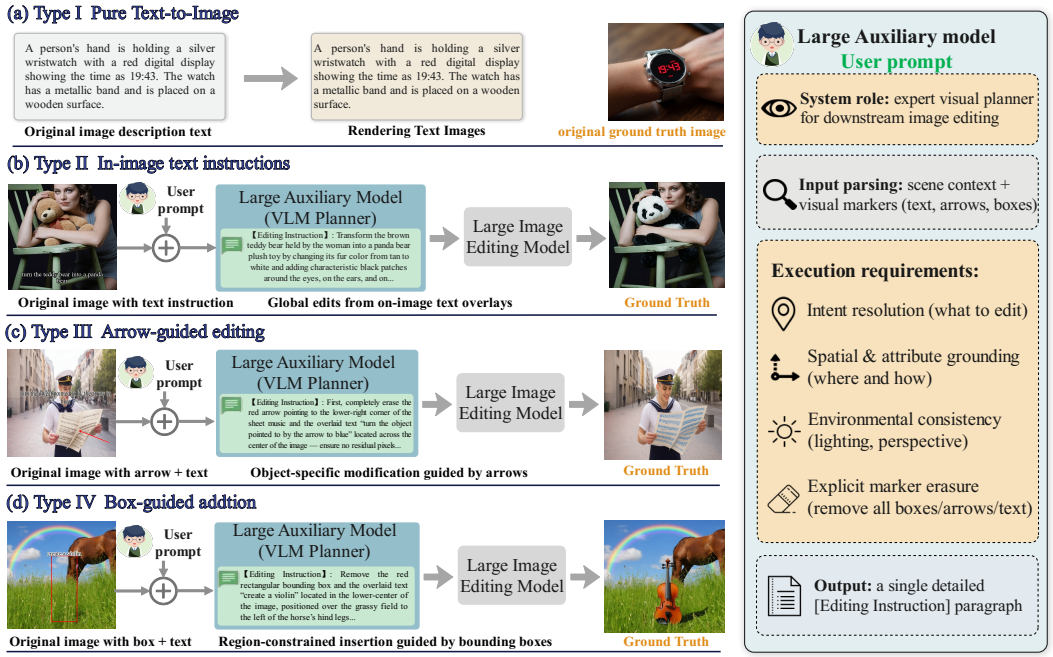


Figure 2: **Typology of Visual Instructions.** Our data engine constructs four distinct categories of instruction pairs: (I) Pure text-to-image, where descriptive text rendered on a blank canvas serves as the input to generate the corresponding realistic image; (II) In-image textual instructions for global edits via text overlays; (III) Arrow-guided modification for precise object-specific changes; and (IV) Box-guided addition for region-specific insertion. For the complex visual markers (Types II-IV), a large auxiliary model acts as a spatial planner to interpret user intent and ensure artifact removal, while a base image editor executes the synthesized instructions to generate the clean ground truth.

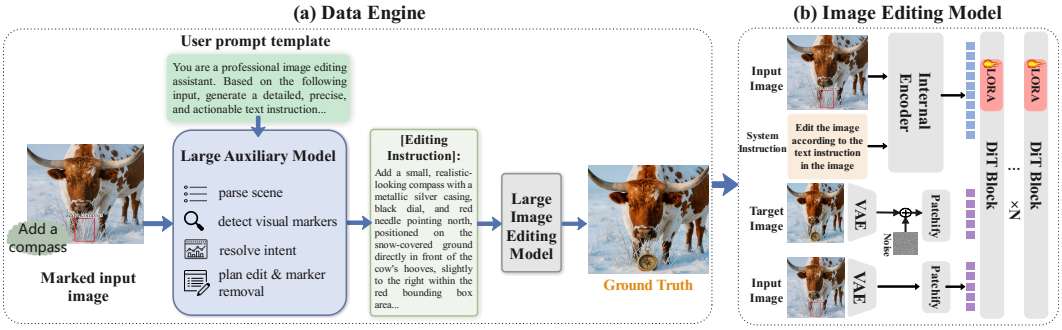


Figure 3: **Overview of the Siphon Framework.** *Stage 1 (Elicitation via Data Synthesis)*: We take a *Large Auxiliary Vision-Language Model* as the spatial planner to interpret visual markers and generate refined text instructions, paired with a *Large Image Editing Model* to generate clean ground-truth images (we instantiate them with Qwen-VL-Max and Qwen-Image-Edit/LongCat-Image in this work). *Stage 2 (Internalization via Fine-Tuning)*: We freeze the backbone editing model, and train lightweight LoRA adapters to learn the end-to-end mapping from visually marked input images to edited results. Our framework separates the choice of auxiliary VLM from the editing backbone; we validate it on two editing architectures.

encoded by the markers. It then produces a refined textual description that explicitly specifies both the desired edit and the removal of all visual instructions.

Formally, we obtain a refined instruction

$$T_{\text{refined}} = \text{VLM}(I_{\text{marked}}), \tag{2}$$

which is subsequently executed by a text-driven image editor to generate the clean target image

$$I_{\text{edited}} = \text{EditModel}(I_{\text{src}}, T_{\text{refined}}). \tag{3}$$

By explicitly specifying artifact removal as part of the VLM-generated editing instruction, the synthesized targets reflect only the intended semantic transformation, with all visual instruction markers excluded.

3.2 Internalizing Visual Instruction Following

While the elicitation stage provides paired supervision, the internalization stage transfers this externally elicited competence into a single image editing model. Our objective is to shift the conditioning paradigm from text-based to vision-based, such that the model learns to interpret and execute visual instructions directly from pixels.

Conventional image editing models learn a conditional distribution $p(I_{\text{edited}} \mid I_{\text{src}}, T)$, where T is a textual prompt. In contrast, we remove sample-specific textual conditioning and treat the marked image I_{marked} as the sole source of editing intent. The backbone's fixed default text input is identical across samples during training and vision-only inference, so it provides no sample-specific instructions.

Our internalization paradigm is backbone-agnostic. In our primary implementation, we adopt recent state-of-the-art diffusion transformers, specifically Qwen-Image-Edit and the highly efficient LongCat-Image, as our backbones. To preserve their pretrained generative priors while injecting visual instruction-following capability, we freeze all backbone parameters and introduce Low-Rank Adaptation (LoRA) modules [14] into the attention layers. Specifically, LoRA adapters are applied to the Q, K, V, and output projections: $W' = W + \alpha BA$, where $A \in \mathbb{R}^{r\times d}$ and $B \in \mathbb{R}^{d\times r}$ are learnable low-rank matrices with $r \ll d$. With all backbone parameters frozen, learning is concentrated in the LoRA adapters, which introduce an inductive bias that steers the attention layers toward instruction-relevant visual cues and their corresponding semantic effects.

Training follows the standard Flow Matching objective, which is the native formulation for our backbone models (Qwen-Image-Edit and LongCat-Image). Given a noisy latent z_t interpolated from the target image latent z_1 and Gaussian noise z_0 via the forward process $z_t = tz_1 + (1-t)z_0$ ($t \in [0, 1]$), and the latent encoding of the marked conditioning image z_{marked} as the sole condition, we optimize:

$$\mathcal{L} = \mathbb{E}_{t,z_0,z_1}\left[\|v_\theta(z_t, t, z_{\text{marked}}) - (z_1 - z_0)\|^2\right]. \tag{4}$$

Here, v_θ denotes the velocity field predicted by our model, and the regression target $(z_1 - z_0)$ is the target velocity along the linear interpolation path. Since the target image z_1 is free of markers while the conditioning image z_{marked} contains them, the objective implicitly enforces an adaptive behavior: the model must remove visual instructions while executing the semantic edit they represent.

3.3 Two-Stage Training with Text-as-Image Forcing

Directly training the model to simultaneously recognize rendered text and execute semantic edits is challenging, as it requires disentangling low-level text recognition from high-level instruction execution. To address this, we adopt a two-stage training strategy that progressively aligns visual perception with semantic editing.

In the first stage, termed Text-as-Image Forcing (TAIF), the model is trained exclusively on Type I samples with text rendered as images, establishing a robust visual-text grounding. In the second stage, we fine-tune the model on the full mixed dataset, allowing the model to transfer this grounding to more abstract visual markers, leading to more stable training and improved instruction adherence. We emphasize that TAIF is used as a visual-text grounding curriculum rather than as an argument that all user instructions should be rendered inside images.

4 Experiments

4.1 Experimental Setup

Implementation and Dataset. We take Qwen-Image-Edit (DiT-based) as our primary backbone, and validate the architecture-agnostic nature of Siphon on the efficient LongCat-Image (6B) [46, 28]. For both models, we freeze full backbone weights and inject LoRA adapters [14] into the Q, K, V, O projection layers of all attention blocks, with rank $r = 64$ to balance performance and parameter efficiency. We adopt a two-stage training pipeline (see Appendix for full hyperparameters), and construct a synthetic training set of 100k samples (20k Type I pure text-to-image, 80k Types II–IV visual markers) and a held-out 4k test set (1k per instruction type), with no train-test overlap and consistent distribution of edit operations (addition/removal/modification) for unbiased evaluation.

Baselines and Fair Comparison Protocol. We compare against two categories of state-of-the-art models: (1) Specialized Editing Models: InstructPix2Pix [4], Qwen-Image-Edit (Base), LongCat-Image (Base); (2) Unified Multimodal Foundation Models: ICEdit [58], Step1X-Edit [24], BAGEL [9], OmniGen2 [47], and UniWorld-V2 [19]. We also implement a strong pipeline baseline (Qwen-VL-Max + Qwen-Edit), where the VLM translates visual markers into textual prompts for the base editor. For strict fairness, all prompt-conditioned baselines are fed a unified prompt that explicitly requests both visual grounding and marker removal: "*Use the visual annotations and any overlaid instruction text in the image to perform the requested edit. Remove all artificial visual annotations.*" This ensures that marker-removal performance is not inflated by prompt asymmetry.

Evaluation Metrics. We use a three-part evaluation system to cover core performance, backward compatibility, and user experience. For visual-only editing tasks, we adopt CLIP-GT (semantic alignment to ground truth), CLIP-Text (Oracle, caption-based intent alignment), PSNR/LPIPS (pixel and perceptual similarity to the reference), Marker Removal Rate (MRR, via VLM-as-Judge [34]), and Box-IoU (spatial precision for box-guided tasks). To verify no catastrophic forgetting of standard text-editing capabilities, we evaluate on public benchmarks GEdit-Bench [24, 42] and CEdit-Bench [28, 27]. We also conduct a blind user study with 30 volunteers, who rate 400 randomized samples on a 1–5 Likert scale for Spatial Accuracy, Marker Removal, and Visual Quality. Full metric definitions and implementation details are provided in Appendix.

4.2 Main Results

4.2.1 Quantitative Results

Table 1 presents the quantitative comparison under the fair marker-removal protocol. Siphon consistently outperforms prompt-conditioned baselines across both backbones. Siphon w/ Qwen achieves 90.41 CLIP-GT, improving over Qwen-Edit Base by 16.07 points, while Siphon w/ LongCat reaches 91.05, improving over LongCat-Edit Base by 15.93 points and the strongest non-pipeline baseline, OmniGen2, by 15.14 points. Siphon w/ LongCat also obtains the best CLIP-Text score of 34.62, Box-IoU of 0.84, and MRR of 98.6%. These results suggest that Siphon internalizes both visual grounding and marker removal, rather than relying on prompt wording or an external VLM planner.

We further validate the VLM-based MRR metric with human annotations on 400 randomly sampled outputs across methods and instruction types. Three annotators independently judge whether any artificial marker remains, and we take the majority vote as the human label. GPT-4o agrees with majority human labels on 91.5% of samples with Cohen's $\kappa = 0.82$, indicating strong agreement.

Backward compatibility. We further verify that internalizing visual instructions does not cause catastrophic forgetting of native text-driven editing. Following the official protocols of GEdit-Bench [42] and CEdit-Bench [27], both Siphon variants preserve the performance of their respective base models, with consistent modest gains in our evaluation. Specifically, Siphon w/ Qwen improves G_O from 7.54 to 7.84 on GEdit-Bench and from 7.49 to 7.80 on CEdit-Bench, while Siphon w/ LongCat improves G_O from 7.62 to 7.89 and from 7.66 to 7.93, respectively. Full results are reported in Appendix E.1.

Spatial disambiguation. To test whether visual markers provide information beyond being another carrier of text, we further evaluate a separated-text setting and a hard multi-object disambiguation setting. Even when the text instruction is provided separately, adding visual markers improves target accuracy from 72.4% to 79.6% and reduces wrong-object edits from 18.9% to 10.8%. In hard

Table 1: Quantitative comparison under the fair marker-removal protocol. All prompt-conditioned baselines are explicitly instructed to follow visual annotations and remove artificial markers from the final image. The Pipeline Baseline uses Qwen-VL-Max to translate visual markers into text prompts for the base editor. The test set contains 1k examples per instruction type (4k total); see Sec. 3.1 and Appendix D.1 for construction and splits. Box-IoU uses the box-guided subset. CLIP-GT is scaled by 100.

Method	CLIP-GT ↑	CLIP-Text ↑	Box-IoU ↑	PSNR(dB) ↑	LPIPS ↓	MRR(%) ↑
InstructPix2Pix	62.57	27.50	0.35	10.30	0.292	72.5
ICEdit	67.82	29.10	0.42	11.20	0.271	77.6
Step1X-Edit	69.35	29.80	0.51	11.60	0.258	79.2
BAGEL	71.58	30.50	0.58	12.30	0.243	81.4
UniWorld-V2	72.64	31.20	0.62	12.70	0.236	82.8
OmniGen2	75.91	32.60	0.72	13.40	0.212	85.7
Qwen-Edit (Base)	74.34	32.14	0.68	13.96	0.194	84.3
LongCat-Edit (Base)	75.12	32.45	0.70	14.10	0.187	85.1
Pipeline Baseline(VLM+Editor)						
Qwen-Edit + Qwen-Max	90.12	34.01	0.78	14.52	0.141	96.5
LongCat-Edit + Qwen-Max	90.68	34.32	0.80	14.85	0.136	96.9
Ours (Siphon) w/ Qwen	90.41	34.28	0.82	15.79	0.124	98.2
Ours (Siphon) w/ LongCat	**91.05**	**34.62**	**0.84**	**16.05**	**0.117**	**98.6**

multi-object scenes with 2–10 similar objects, Siphon w/ LongCat reaches 92.4% target accuracy and only 2.6% wrong-object edits, compared with 64.5% and 23.8% for the base editor. These results show that visual markers contribute spatial grounding rather than merely replacing text.

4.2.2 Subjective User Evaluation

Table 2 shows the user study results, where both of our Siphon variants scored significantly higher (> 4.0) across all dimensions. Specifically, for Spatial Accuracy, Siphon w/ Qwen averaged 4.2 and Siphon w/ LongCat reached 4.4, representing a substantial improvement over the strongest baseline (3.6); this confirms that visual markers resolve the referential ambiguity inherent in text prompts. Regarding Marker Removal, users rated our removal effects at 4.5 and 4.6 respectively, significantly better than the base models ($\sim$3.3), noting "clean outputs without residuals." Driven by this precise control and clean removal, Siphon w/ LongCat achieved a Visual Quality score of 4.5. Paired t-tests confirm these differences are statistically significant ($p < 0.05$).

Table 2: User Study Results (1–5 Likert Scale). All prompt-conditioned baselines follow the same marker-removal prompt protocol as in Table 1. Data are presented as Mean $\pm$ Std. Our methods achieve significant improvements in spatial grounding and artifact removal across both backbones.

Method	Spatial Acc.	Marker Removal	Visual Quality	Average
InstructPix2Pix	2.8 ± 0.6	2.7 ± 0.7	2.9 ± 0.5	2.8 ± 0.6
BAGEL	3.3 ± 0.5	3.5 ± 0.6	3.4 ± 0.5	3.4 ± 0.5
Step1X-Edit	3.5 ± 0.4	3.6 ± 0.5	3.5 ± 0.4	3.5 ± 0.4
OmniGen2	3.6 ± 0.4	3.7 ± 0.5	3.6 ± 0.4	3.6 ± 0.4
Qwen-Edit (Base)	3.1 ± 0.5	3.3 ± 0.6	3.2 ± 0.5	3.2 ± 0.5
LongCat-Edit (Base)	3.2 ± 0.5	3.3 ± 0.5	3.3 ± 0.5	3.3 ± 0.5
Ours (Siphon) w/ Qwen	$\mathbf{4.2 \pm 0.4}$	$\mathbf{4.5 \pm 0.3}$	$\mathbf{4.3 \pm 0.4}$	$\mathbf{4.3 \pm 0.4}$
Ours (Siphon) w/ LongCat	$\mathbf{4.4 \pm 0.3}$	$\mathbf{4.6 \pm 0.3}$	$\mathbf{4.5 \pm 0.3}$	$\mathbf{4.5 \pm 0.3}$

Per-Instruction-Type Breakdown. Siphon consistently improves across all four instruction types, with the largest gain on Type I pure text-rendering (≥ 0.92 vs. baselines ≈ 0.30), where baselines fail to interpret rendered text as instructions. Full per-type results are reported in Appendix E.2.

4.3 Qualitative Results

Fig. 4 visually demonstrates our model's superiority across diverse instruction types. In the first column for Implicit OCR (Type I), given a text description of a rifle, the baseline erroneously renders text characters, whereas ours generates a photorealistic rifle matching the description. For Global Harmony (Type II) in the second column (e.g., "Add rainbow"), the baseline fails to interpret the command, treating the text overlay as texture and generating artifacts; in contrast, ours correctly interprets the semantic intent, placing a natural rainbow in the sky while maintaining lighting consistency. Regarding Referential Resolution (Type III) in the sixth column, the instruction requires changing the object pointed to by an arrow to "pink with purple and blue stripes." While the baseline

fails to execute this complex attribute modification, our model correctly resolves the spatial reference and successfully generates the specific color pattern. Finally, for Precise Spatial Control (Type IV) in the third column (e.g., "Insert a plum"), the baseline fails to generate the target object. Conversely, ours successfully inserts the plum strictly within the defined boundary and simultaneously erases the box marker. Furthermore, to demonstrate Siphon's robustness in real-world interactive scenarios, we evaluate its zero-shot generalization to free-form, hand-drawn annotations (e.g., irregular boxes and squiggly arrows). See Appendix G for hand-drawn examples.

Figure 4: **Qualitative Comparison on vision-only Editing.** We compare Siphon against state-of-the-art baselines across different instruction types (Columns). The rows correspond to: (1) Input images with visual markers; (2) OmniGen2 [47]; (3) Qwen-Image-Edit (Base); (4) **Ours (Siphon) w/ Qwen**; (5) LongCat-Edit (Base); (6) **Ours (Siphon) w/ LongCat**; and (7) Ground Truth. Observations: All baseline models (Rows 2, 3, and 5) struggle to ground the visual instructions, often misinterpreting markers as texture artifacts (e.g., painting the box into the object) or ignoring the spatial constraints entirely. In contrast, our Siphon framework applied to both backbones (Rows 4 and 6) successfully interprets and internalizes the visual intent, precisely executing the edits while simultaneously removing the visual markers to yield clean, photorealistic results.

4.4 Ablation Studies

Ablation 1: Text-as-Image Forcing (TAIF). To investigate the necessity of our two-stage curriculum, we introduce Text-as-Image Forcing (TAIF), an explicit pre-alignment phase where the model reconstructs natural images solely from typographically rendered text prompts (Type I data). This encourages the model to associate pixel-level glyphs with semantic concepts while its encoders remain frozen.

We compare three training configurations: 1) Joint Training (w/o TAIF), where the pretrained model is adapted directly on the mixed dataset ($\mathcal{D}_{mix}$ = Type I $\cup$ Type II-IV) for 100k steps; 2) TAIF-Only (Zero-Shot), where the model is trained only on Type I data and directly evaluated on Type II-IV tasks; and 3) Sequential TAIF (Ours), where the model first undergoes TAIF for 20k steps and then fine-tunes on $\mathcal{D}_{mix}$ for 80k steps.

Quantitative Analysis. As detailed in Appendix E.5, Joint Training performs sub-optimally across both backbones, yielding an average score of only $\sim$0.67, while TAIF-Only achieves high Type-I

performance but fails to interpret abstract visual markers such as arrows and boxes. Sequential TAIF achieves the best overall performance on both Qwen-Edit and LongCat-Edit, indicating that visual-text grounding provides an effective curriculum before learning spatial visual instructions.

Additional ablations. We further ablate the LoRA rank and the VLM-guided synthesis pipeline. Rank 64 provides the best efficiency–performance trade-off, while VLM-guided synthesis substantially improves spatial grounding and marker removal over naive synthesis. Full results are reported in Appendix E.4.

5 Conclusion

We introduced Siphon, a framework that internalizes externally elicited visual instruction-following competence into a single image editing model. By eliminating reliance on external VLM modules at inference time, Siphon enables end-to-end *vision-only* editing, where users specify both edit intent and spatial constraints directly through on-image text and visual markers.

Our approach combines VLM-guided data synthesis with parameter-efficient adaptation, enabling the model to interpret visual instructions as executable signals and remove instruction artifacts during generation. Extensive experiments show that Siphon substantially improves instruction adherence, spatial controllability, and marker removal over strong text-driven baselines, while matching VLM-assisted pipelines without their additional VLM-planning latency, external API dependence, and cross-model error propagation.

Acknowledgments and Disclosure of Funding

This work was supported by the Science and Technology Planning Project of Guangdong Province (Grant No. 2024B1212020007) for the Guangdong Provincial Key Laboratory of High-end Integrated Circuit Design and Integration Technology (Sun Yat-Sen University), and by the Shenzhen Science and Technology Program (Grant No. KJZD20230923114806013). The authors declare no other competing interests.

References

[1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[2] James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, et al. Improving image generation with better captions. Technical report, OpenAI, 2023.

[3] Black Forest Labs, Stephen Batifol, Andreas Blattmann, Frederic Boesel, et al. Flux.1 kontext: Flow matching for in-context image generation and editing in latent space, 2025.

[4] Tim Brooks, Aleksander Holynski, and Alexei A Efros. Instructpix2pix: Learning to follow image editing instructions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18392–18402, 2023.

[5] Mu Cai, Haotian Liu, Siva Karthik Mustikovela, Gregory P. Meyer, Yuning Chai, Dennis Park, and Yong Jae Lee. ViP-LLaVA: Making large multimodal models understand arbitrary visual prompts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12914–12923, 2024.

[6] Jingye Chen, Yupan Huang, Tengchao Lv, Lei Cui, Qifeng Chen, and Furu Wei. Textdiffuser-2: Unleashing the power of language models for text rendering. In *European Conference on Computer Vision (ECCV)*, pages 386–402. Springer, 2024.

[7] Jiuhai Chen, Zhiyang Xu, Xichen Pan, Yushi Hu, Can Qin, Tom Goldstein, Lifu Huang, Tianyi Zhou, Saining Xie, Silvio Savarese, et al. Blip3-o: A family of fully open unified multimodal models-architecture, training and dataset. *arXiv preprint arXiv:2505.09568*, 2025.

[8] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. Sharegpt4v: Improving large multi-modal models with better captions. In *European Conference on Computer Vision*, pages 370–387. Springer, 2024.

[9] Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, et al. Emerging properties in unified multimodal pretraining. *arXiv preprint arXiv:2505.14683*, 2025.

[10] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning (ICML)*, 2024.

[11] Lijie Fan, Luming Tang, Siyang Qin, Tianhong Li, Xuan Yang, et al. Unified autoregressive visual generation and understanding with continuous tokens. *arXiv preprint arXiv:2503.13436*, 2025.

[12] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross attention control. *arXiv preprint arXiv:2208.01626*, 2022.

[13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[15] jackyhate. text-to-image-2m: A high-quality, diverse text-to-image training dataset. `https://huggingface.co/datasets/jackyhate/text-to-image-2M`, 2024.

[16] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, pages 19730–19742, 2023.

[17] Minglei Li, Peng Ye, Yongqi Huang, Lin Zhang, Tao Chen, et al. Adapter-x: A novel general parameter-efficient fine-tuning framework for vision. *arXiv preprint arXiv:2406.03051*, 2024.

[18] Xiang Li, Kai Qiu, Hao Chen, Jason Kuen, Zhe Lin, Rita Singh, and Bhiksha Raj. Controlvar: Exploring controllable visual autoregressive modeling. *arXiv preprint arXiv:2406.09750*, 2024.

[19] Zongjian Li, Zheyuan Liu, Qihui Zhang, Bin Lin, Shenghai Yuan, et al. Uniworld-v2: Reinforce image editing with diffusion negative-aware finetuning and mllm implicit feedback. *arXiv preprint arXiv:2510.16888*, 2025.

[20] Bin Lin, Zongjian Li, Xinhua Cheng, Yuwei Niu, Yang Ye, Xianyi He, et al. Uniworld: High-resolution semantic encoders for unified visual understanding and generation. *arXiv preprint arXiv:2506.03147*, 2025.

[21] Dongyang Liu, Renrui Zhang, Longtian Qiu, Siyuan Huang, Weifeng Lin, Shitian Zhao, Shijie Geng, Ziyi Lin, Peng Jin, Kaipeng Zhang, et al. Sphinx-x: Scaling data and parameters for a family of multi-modal large language models. *arXiv preprint arXiv:2402.05935*, 2024.

[22] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[23] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, and Hang Su. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *ECCV*, 2024.

[24] Shiyu Liu, Yucheng Han, Peng Xing, Fukun Yin, Rui Wang, et al. Step1x-edit: A practical framework for general image editing. *arXiv preprint arXiv:2504.17761*, 2025.

[25] Yiting Lu, Jiakang Yuan, Zhen Li, Shitian Zhao, Qi Qin, Xinyue Li, Le Zhuo, Licheng Wen, Dongyang Liu, Yuewen Cao, et al. Omnicaptioner: One captioner to rule them all. *arXiv preprint arXiv:2504.07089*, 2025.

[26] Yiyang Ma, Xingchao Liu, Xiaokang Chen, Wen Liu, Chengyue Wu, et al. Janusflow: Harmonizing autoregression and rectified flow for unified multimodal understanding and generation. *arXiv preprint arXiv:2411.07975*, 2024.

[27] Meituan LongCat Team. CEdit-Bench: A Fine-Grained Image Editing Benchmark. `https://huggingface.co/datasets/meituan-longcat/CEdit-Bench`, 2025.

[28] Meituan LongCat Team, Hanghang Ma, Haoxian Tan, Jiale Huang, Junqiang Wu, Jun-Yan He, Lishuai Gao, Songlin Xiao, Xiaoming Wei, Xiaoqi Ma, Xunliang Cai, Yayong Guan, and Jie Hu. Longcat-image technical report. *arXiv preprint arXiv:2512.07584*, 2025.

[29] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. Sdedit: Guided image synthesis and editing with stochastic differential equations. *arXiv preprint arXiv:2108.01073*, 2021.

[30] Zijie Meng. Decoupling semantics from distortions: Multi-scale two-stream vision-language alignment for AI-generated image quality assessment. *arXiv preprint arXiv:2606.16799*, 2026.

[31] Zijie Meng, Yufei Liu, Chengqian Ma, Zhiyu Li, Jiyuan Liu, Wenhua Nie, Bingcai Wei, Shuqin Chen, Weichen Xu, Jiquan Yuan, and Miao Zhang. OmniDrive: An LLM-choreographed multi-agent world model with unified latent co-compression for multi-view driving video generation. *arXiv preprint arXiv:2606.17536*, 2026.

[32] Ron Mokady, Amir Hertz, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Null-text inversion for editing real images using guided diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6038–6047, 2023.

[33] Chong Mou, Xintao Wang, Liangbin Xie, Jing Zhang, Zhongang Qi, Ying Shan, and Xiaohu Qie. T2i-adapter: Learning adapters to dig out more controllable ability for text-to-image diffusion models. *ArXiv*, abs/2302.08453, 2023.

[34] OpenAI. Hello gpt-4o, May 2024.

[35] Gaurav Parmar, Krishna Kumar Singh, Richard Zhang, Yijun Li, Jingwan Lu, and Jun-Yan Zhu. Zero-shot image-to-image translation. *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

[36] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.

[37] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[38] Yusu Qian, Eli Bocek-Rivele, Liangchen Song, Jialing Tong, Yinfei Yang, Jiasen Lu, Wenze Hu, and Zhe Gan. Pico-Banana-400K: A large-scale dataset for text-guided image editing. *arXiv preprint arXiv:2510.19808*, 2025.

[39] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, et al. SAM 2: Segment anything in images and videos. In *The Thirteenth International Conference on Learning Representations*, 2025.

[40] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[41] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, et al. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

[42] StepFun AI. GEdit-Bench Dataset. `https://huggingface.co/datasets/stepfun-ai/GEdit-Bench`, 2025.

[43] Yuxiang Tuo, Wangmeng Xiang, Jun-Yan He, Yifeng Geng, and Xuansong Xie. Anytext: Multilingual visual text generation and editing. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[44] Xinlong Wang, Xiaosong Zhang, Yue Cao, Wen Wang, Chunhua Shen, and Tiejun Huang. SegGPT: Towards segmenting everything in context. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1130–1140, 2023.

[45] Cong Wei, Zheyang Xiong, Weiming Ren, Xinrun Du, Ge Zhang, and Wenhu Chen. OmniEdit: Building image editing generalist models through specialist supervision. *arXiv preprint arXiv:2411.07199*, 2024.

[46] Chenfei Wu, Jiahao Li, Jingren Zhou, Junyang Lin, et al. Qwen-image technical report. *arXiv preprint arXiv:2508.02324*, 2025.

[47] Chenyuan Wu, Pengfei Zheng, Ruiran Yan, Shitao Xiao, Xin Luo, et al. Omnigen2: Exploration to advanced multimodal generation. *arXiv preprint arXiv:2506.18871*, 2025.

[48] Shitao Xiao, Yueze Wang, Junjie Zhou, Huaying Yuan, Xingrun Xing, et al. Omnigen: Unified image generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 13294–13304, 2025.

[49] Jinheng Xie, Weijia Mao, Zechen Bai, David Junhao Zhang, Weihao Wang, et al. Show-o: One single transformer to unify multimodal understanding and generation. In *The Thirteenth International Conference on Learning Representations*, 2025.

[50] Tianyi Xiong, Xiyao Wang, Dong Guo, Qinghao Ye, Haoqi Fan, Quanquan Gu, Heng Huang, and Chunyuan Li. Llava-critic: Learning to evaluate multimodal models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 13618–13628, 2025.

[51] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[52] Hu Ye, Jun Zhang, Sibo Liu, Xiao Han, and Wei Yang. Ip-adapter: Text compatible image prompt adapter for text-to-image diffusion models. *arXiv preprint arXiv:2308.06721*, 2023.

[53] Yang Ye, Xianyi He, Zongjian Li, Bin Lin, Shenghai Yuan, Zhiyuan Yan, Bohan Hou, and Li Yuan. Imgedit: A unified image editing dataset and benchmark. *arXiv preprint arXiv:2505.20275*, 2025.

[54] Huanyu Zhang, Xuehai Bai, Chengzu Li, Chen Liang, Haochen Tian, Haodong Li, Ruichuan An, Yifan Zhang, Anna Korhonen, Zhang Zhang, Liang Wang, and Tieniu Tan. How well do models follow visual instructions? VIBE: A systematic benchmark for visual instruction-driven image editing. *arXiv preprint arXiv:2602.01851*, 2026.

[55] Kai Zhang, Lingbo Mo, Wenhu Chen, Huan Sun, and Yu Su. Magicbrush: A manually annotated dataset for instruction-guided image editing. *Advances in Neural Information Processing Systems*, 36:31428–31449, 2023.

[56] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3836–3847, 2023.

[57] Renrui Zhang, Jiaming Han, Chris Liu, Peng Gao, Aojun Zhou, et al. Llama-adapter: Efficient fine-tuning of language models with zero-init attention. *arXiv preprint arXiv:2303.16199*, 2023.

[58] Zechuan Zhang, Ji Xie, Yu Lu, Zongxin Yang, and Yi Yang. In-context edit: Enabling instructional image editing with in-context generation in large scale diffusion transformer. *arXiv preprint arXiv:2504.20690*, 2025.

A Prompt Templates for Data Synthesis

Note that Type I (Pure Text-to-Image) does not require a VLM planner for instruction generation, since it is synthesized via deterministic typographic rendering. Its synthesis procedure is described in Sec. C.1. Below, we provide the prompt templates used for the marker-based instruction types (Types II–IV).

To ensure reproducibility, we provide the prompt templates and formatting rules used in our Marker-Data-Engine. The bracketed fields are dynamically populated based on image-specific observations. In our automated pipeline, these fields are populated by the same auxiliary VLM (Qwen-VL-Max) based on the marked input image, requiring no manual human annotation to generate structured editing instructions and ground truths for Types II, III, and IV.

A.1 Prompt for Type II: In-Image Textual Instructions

For images containing directly overlaid textual commands (without explicit markers like arrows or boxes), we used:

System Prompt: In-Image Text Instruction

Role: You are a professional image editing assistant. Based on the following input, generate a detailed, precise, and actionable text instruction that can be directly used by an image editing model.
Input:

- Image content: [Describe the main elements in the image, including subjects, background, lighting, weather, time of day, etc.]
- In-image text: [Extract and describe any visible text written within the image, such as "Add a white horse with a saddle in the background."]

Requirements:

1. Clearly identify what object(s) should be added, removed, or modified based on the in-image text.
2. Determine the most plausible location for the new object(s), using relative spatial references (e.g., "in the background behind the right character", "to the left of the tree") or depth cues.
3. Describe visual attributes in detail: color, size, type, orientation, material, style (realistic, cinematic, etc.), and whether it should appear partially occluded or fully visible.
4. Ensure environmental consistency: match lighting direction, shadow casting, perspective, scale, and texture with the original scene.
5. Explicitly instruct the model to remove any visible text annotations from the final image.
6. Use fluent natural language but be highly specific—avoid ambiguity so the editing model can execute accurately.

Output format:
[Editing Instruction]: [A single, coherent, and comprehensive sentence or paragraph describing the edit.]

A.2 Prompt for Type III: Arrow-Guided Instructions

For tasks involving attribute modification or replacement indicated by arrows (Arrow + Text), we used:

System Prompt: Arrow-Guided Editing

Role: You are a professional image editing assistant. Based on the following input, generate a detailed, precise, and actionable text instruction that can be directly used by an image editing model.
Input:

- Image content: [Describe the main elements in the image, including subjects, background, lighting, weather, time of day, etc.]

- Visual annotations: [Describe any arrows, bounding boxes, or text labels present. Specify the object pointed to by the arrow and its approximate location. Also quote the exact textual instruction written in the image.]

Requirements:

1. Clearly identify the target object indicated by the arrow (e.g., "the red hoodie").
2. Determine the exact edit type based on the quoted text: swap, replace, change color, delete, add, etc.
3. Describe the new object in detail: color, material, style, size, orientation, and how it should fit into the scene (e.g., "light blue denim button-up shirt with two chest pockets").
4. Ensure environmental consistency: match lighting direction, shadows, fabric texture, perspective, and scale with the original image.
5. If the edit involves replacing an object, explicitly instruct the model to remove the original object and seamlessly integrate the new one without visible seams, distortions, or artifacts.
6. Explicitly and unambiguously instruct the model to first erase all artificial visual annotations before performing the main edit. This includes:
   - The arrow (specify its color and direction if known),
   - The adjacent text label (quote the exact words),
   - Any other non-original markings such as boxes, lines, or highlights.

   Ensure these elements are completely removed with no residual pixels, halos, blurring, or background mismatch—the final image must appear as if no annotations were ever present.
7. Use fluent natural language but be highly specific—avoid ambiguity so the editing model can execute accurately.

Output format:
[Editing Instruction]: [A single, coherent, and comprehensive sentence or paragraph describing the edit. Begin by instructing the removal of all annotations, then describe the main edit in full detail.]

A.3 Prompt for Type IV: Bounding Box Instructions

For tasks requiring object addition or placement within a specific region (Box + Text), we used:

System Prompt: Bounding Box Generation

Role: You are a professional image editing assistant. Based on the following input, generate a detailed, precise, and actionable text instruction that can be directly used by an image editing model.
Input:

- Image content: [Describe the main elements in the image, including subjects, background, lighting, weather, time of day, etc.]
- Visual annotations: [Describe any human-added markings such as red bounding boxes, arrows, handwritten or typed text labels, and their spatial relationship to objects in the scene]

Requirements:

1. Clearly specify what object should be added, removed, or modified (e.g., "a red toy car").
2. Precisely indicate its location using relative spatial references (e.g., "slightly in front of the elephant's left front foot") or approximate coordinates if appropriate.
3. Describe visual attributes in detail: color, size, type, orientation, material, style (e.g., realistic vs. cartoon), etc.
4. Ensure environmental consistency: match lighting direction, shadow casting, ground texture, perspective, and scale with the original scene.
5. Use fluent natural language, but be highly specific—avoid ambiguity so the editing model can execute the change accurately.
6. If the image contains artificial annotations (e.g., red bounding boxes, text labels like 'create a violin'), explicitly include instructions to remove or erase them from the final edited image.

Output format:
[Editing Instruction]: [A single, coherent, and comprehensive sentence or paragraph describing the edit.]

B VLM-as-a-Judge Prompt for MRR Evaluation

To assess marker removal at scale, we use an external VLM (GPT-4o) as an auxiliary automatic evaluator. GPT-4o is not used in our training pipeline or in the construction of the target images for the main experiments. We therefore treat this score as a *proxy* for visible marker residue, and interpret it together with user study results and qualitative comparisons rather than as a standalone perceptual metric.

For each edited output image, the evaluator is asked to determine whether any artificial visual markers remain visible in the final result. The judgment is based only on the edited output image. We use the following binary prompt:

Prompt: Automatic Evaluation for Marker Residue

Role: You are an image evaluator.
Task: Examine the provided image and determine whether any artificial visual markers are still visible in the scene.
Artificial visual markers include:

- Bounding boxes or partial box traces,
- Arrows or pointer-like annotation traces,
- Overlaid text, letters, or instruction words,
- Visible remnants of such annotations (e.g., faint colored edges or residual strokes).

Output requirement: Answer only with **Yes** or **No**. **Yes**: at least one artificial marker or visible residue remains. **No**: no artificial marker or visible residue is observed.

We define the Marker Removal Rate (MRR) as the percentage of test samples for which the evaluator outputs **No**. The same evaluation prompt and decoding setting are used for all compared methods.

B.1 Human Validation of Marker Removal Rate

To validate the reliability of our automatic Marker Removal Rate (MRR), we compare GPT-4o judgments with human annotations. We randomly sample 400 output images across evaluated methods and instruction types to ensure coverage. For each image, three annotators independently answer a binary question: *"Are there any visible artificial visual annotations remaining in the final image, including arrows, boxes, circles, or overlaid text?"* We take the majority vote as the human label.

GPT-4o judgments agree with human majority labels on 91.5% of samples, with Cohen's $\kappa = 0.82$. This indicates strong agreement between the automatic judge and human perception. We therefore use GPT-4o-based MRR for large-scale evaluation, while reporting this human validation to calibrate the automatic metric.

C Implementation Details and Efficiency Analysis

C.1 Deterministic Typographic Rendering (Type I Data)

To construct the Pure Text-to-Image (Type I) dataset, we utilized the Text-to-Image-2M [15] dataset as a source. We developed a robust rendering pipeline to convert textual prompts into visual input images. The pipeline consists of two stages:

1. **Data Filtering:** We filtered samples with prompts exceeding 50 words to ensure legibility and relevance.
2. **Adaptive Rendering:** We implemented a Python-based rendering engine using the Pillow library. The engine features an **adaptive font sizing algorithm** (binary search) to maximize the text visibility within a 1024×1024 canvas while maintaining margins. It supports intelligent line wrapping and handles complex tokenization (e.g., camelCase splitting) to prevent word truncation.

This programmatic approach generated 20,000 high-quality pairs where the input is a white image containing black text (the prompt), and the target is the corresponding ground-truth natural image. This explicitly trains the model to map rendered text glyphs to semantic visual concepts.

C.2 Practical Use Cases of Visual Instruction Types

Each visual instruction type corresponds to a distinct real-world interaction pattern. Potential Type I applications include visual-text inputs such as photographed descriptions, design briefs, or notes; these broader uses are not quantitatively evaluated here. A conventional system would require OCR or manual transcription before sending the extracted text to a text-to-image model, whereas our formulation directly uses the visual text within the same vision-only interface. We use deterministic typographic rendering to construct Type I data because it provides scalable and controllable supervision, but the intended use case is broader than synthetic blank-canvas prompts.

Type II supports short on-canvas commands for global edits, such as changing weather, lighting, style, or scene-level attributes. Type III uses arrows to disambiguate the target object in local edits when multiple similar objects are present. Type IV uses bounding boxes to specify the desired region for object insertion or replacement. Thus, visual text is not meant to replace standard text prompts for long instructions; it covers naturally visual text inputs and also provides a grounding curriculum for more spatially grounded annotations.

C.3 Inference Efficiency Analysis

We measured the average inference time per image on a single NVIDIA A100 (80GB) GPU. The VLM-assisted pipeline introduces a mandatory planning stage before diffusion generation. In our implementation, the external VLM planner takes approximately **30 seconds** per image to perform visual grounding, spatial reasoning, and autoregressive prompt generation.

The diffusion editor itself remains the dominant computational bottleneck. In our latest measurements, the base diffusion editor takes approximately **90 seconds** per image on LongCat-Image and about **150 seconds** on Qwen-Image-Edit. The full VLM-assisted pipeline therefore requires approximately **120 seconds** and **180 seconds**, respectively. Since the lightweight LoRA weights in Siphon can be merged into the backbone linear layers before inference, our internalized model bypasses the planning stage and runs at essentially the same cost as the base editor.

This corresponds to a **25%** end-to-end latency reduction on LongCat-Image (120s $\rightarrow$ 90s) and a **16.7%** reduction on Qwen-Image-Edit (180s $\rightarrow$ 150s). We emphasize that our method does not accelerate the diffusion denoising process itself; rather, it removes the auxiliary VLM-planning stage, external API dependence, and associated cross-model error propagation. The latency saving should therefore be interpreted jointly with these reliability and deployability benefits, rather than as a standalone speedup claim.

C.4 Cached-Latent Training Implementation

Our pretrained backbones (Qwen-Image-Edit and LongCat-Image) are originally text-conditioned image editors. In our default vision-only setting, no *sample-specific* textual instruction is provided during either training or inference. The only exception is the diagnostic separated-text evaluation in Appendix E.3, where we feed the separated instruction through the backbone's original text channel without retraining. To preserve compatibility with the frozen pretrained interface, we keep the default text input required by the backbone unchanged across all samples. Since this text is identical for every example, it carries no instance-level information about the target object, target region, edit type, or desired attributes. Under this formulation, all task-relevant editing intent must be inferred from the visually annotated image I_{marked}.

For training efficiency, we adopt a cached-latent implementation (see Fig. 5). We first preprocess the training set offline and store the conditioning features together with the VAE latents of the marked input image and edited target image. During optimization, the model directly reads these cached representations, which avoids repeated encoder computation and reduces memory overhead. We then freeze the backbone parameters and train only the LoRA layers in the DiT backbone. This design improves training throughput while keeping the conditioning setup consistent with our formulation, where the visually annotated image serves as the only source of sample-specific instruction.

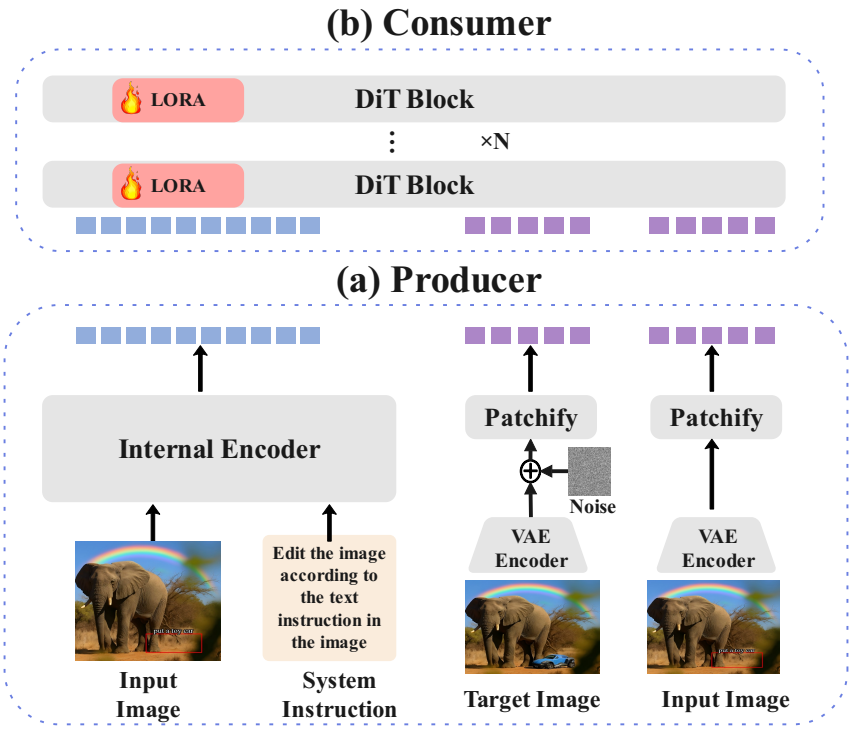


Figure 5: **Overview of the efficient two-stage training pipeline. Bottom (Stage 1: Offline Caching):** We preprocess each training sample with frozen encoders to build an offline cache. Specifically, the marked input image and edited target image are encoded by the VAE into visual latents, while the marked input image is also processed by the frozen internal encoder to produce the conditioning features used for training. **Top (Stage 2: LoRA Fine-Tuning):** We train only the LoRA layers in the DiT backbone by directly consuming the cached conditioning features and visual latents, thereby bypassing redundant encoding steps while ensuring that no sample-specific text instructions are introduced.

D Detailed Experimental Setup

D.1 Full Training Implementation Details

Training is conducted on NVIDIA A100 (80GB) GPUs using the proposed cached-latent training pipeline (see Sec. C.4), in which only lightweight LoRA parameters are optimized while all backbone weights remain frozen. For both stages, we employ the AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay=1×10^{-2}, $\epsilon = 1 \times 10^{-8}$) and `bf16` mixed precision. The input resolution is set to 1024×1024 with dynamic aspect ratio bucketing (`mar_1024`), and the per-GPU batch size is set to 8. Gradient clipping is applied at 1.0.

In practice, the training footprint of the proposed two-stage pipeline is moderate, since we optimize only lightweight LoRA parameters while freezing all backbone weights, and further reduce overhead via cached-latent training. Compared with full-parameter fine-tuning of large diffusion backbones, this substantially lowers both memory usage and overall training cost. The same implementation is also amenable to multi-GPU training when additional hardware is available.

1. **Phase 1 (Text-as-Image Forcing)**: The models are trained exclusively on the 20k Type I samples to learn implicit Optical Character Recognition (OCR) capabilities. This stage is trained for 20k steps with a learning rate of 2×10^{-4} using a constant learning rate scheduler with a 100-step warmup.
2. **Phase 2 (Mixed Fine-Tuning)**: We fine-tune the models for 80k steps on the full 100k mixed dataset (comprising 20k Type I and 80k Types II–IV samples), initialized with Phase

1 weights. To preserve pre-aligned features and prevent catastrophic forgetting, we lower the learning rate to 5×10^{-5} with a 300-step warmup.

LoRA adapters are injected into the Query, Key, Value, and Output projection layers of all attention blocks for both backbones, with rank $r = 64$.

Evaluation Details (Box-IoU Configuration): To ensure reproducibility for the Box-IoU metric calculations (Type IV tasks), we utilize the official pre-trained `groundingdino-swint-ogc` weights for Grounding DINO. The box threshold is set to 0.3 and the text threshold is set to 0.25. If the target object is completely missing (no bounding box detected above the threshold), the Box-IoU for that sample is strictly recorded as 0.

Detailed Data Distribution. To ensure transparency and reproducibility, we detail the exact distribution of our synthetic dataset across the four instruction types. The training set consists of 100,000 samples in total. We allocate 20,000 samples to Type I (Pure Text-to-Image) to establish the foundational visual literacy during the Text-as-Image Forcing (TAIF) phase. This allocation is chosen as a practical trade-off between establishing stable text-as-image alignment and preserving sufficient training budget for the more complex marker-based tasks. The remaining 80,000 samples involve complex visual markers (Types II–IV). Because Type III (Arrow-Guided Modification) covers the most diverse and complex set of semantic operations (e.g., attribute editing, object swapping, and precise referential resolution), it requires a slightly denser data distribution to converge robustly. Therefore, we assign it a slightly larger proportion. Specifically, the 80,000 visual marker samples are distributed as follows: 25,000 for Type II (In-Image Text), 30,000 for Type III (Arrow-Guided), and 25,000 for Type IV (Box-Guided).

In contrast, to ensure our evaluation remains strictly objective and unbiased across different modalities, the 4,000-sample held-out test set is perfectly balanced. It contains exactly 1,000 samples for each of the four instruction types. The balanced split gives each instruction type equal representation; instruction-length statistics are reported in Appendix I.

D.2 Baseline Model Full Details

All baselines are evaluated using publicly available official implementations or official APIs/pre-trained checkpoints when available.

- **Specialized Editing Models**: InstructPix2Pix [4] is the classic instruction-tuned text-driven editing model; Qwen-Image-Edit (Base) and LongCat-Image (Base) [46, 28] are the raw pre-trained foundation models, evaluated under the generic prompt protocol to test their native ability to interpret visual markers.
- **Unified Multimodal Foundation Models**: ICEdit [58], Step1X-Edit [24], BAGEL [9], OmniGen2 [47], and UniWorld-V2 [19] are large-scale multimodal foundation models with integrated editing capabilities.
- **Pipeline Baseline (VLM-Agent)**: We pair Qwen-VL-Max with each base editor backbone used in the main experiments. Specifically, Qwen-VL-Max first interprets the visual markers and translates them into detailed textual editing prompts (including explicit instructions to remove all markers), which are then executed by either the base Qwen-Image-Edit model or the base LongCat-Image model. This pipeline introduces an additional pre-processing latency of $\approx$30s per image.

D.3 Detailed Metric Definitions and Calculations

Recent work on generated-image assessment distinguishes semantic correspondence from perceptual quality [30].

D.3.1 Core Visual-Only Editing Metrics

- **CLIP-GT**: Cosine similarity between the CLIP image embedding of the generated result and the ground truth (GT) image, measuring semantic alignment. CLIP-GT is reported as

raw cosine similarity in the per-type and ablation tables, and multiplied by 100 in Tables 1 and 5.

- **CLIP-Text**: We use Qwen-VL-Max to generate a concise, accurate caption for the GT image, then calculate the CLIP similarity between this caption and the generated image, to evaluate intent adherence beyond pixel-level alignment.
- **PSNR & LPIPS**: Peak Signal-to-Noise Ratio (PSNR) measures low-level reconstruction quality and background preservation; Learned Perceptual Image Patch Similarity (LPIPS) measures perceptual distance from the reference image.
- **MRR (Marker Removal Rate)**: We adopt a VLM-as-a-Judge paradigm using GPT-4o [34] for zero-shot binary classification to detect any residual artificial visual markers (red boxes, arrows, overlaid text) in the generated results. The exact prompt used is provided in Sec. B. MRR is the percentage of samples where no residual markers are detected.
- **Box-IoU**: Calculated exclusively on the Type IV (Box-Guided) subset. Since our editing models generate pixel-level outputs rather than explicit coordinates, we employ a pre-trained open-vocabulary object detector, Grounding DINO [23], to localize the newly synthesized object. Specifically, we use the target object's semantic name as the text query and extract the detected bounding box from the edited image. If multiple candidate boxes are returned, we select the one with the highest detection confidence for evaluation. For the text query, we strictly extract the head noun of the target object (e.g., 'dog', 'plum'). Complex attributes and adjectives are stripped to ensure the open-vocabulary detector focuses purely on object localization. We then compute the Intersection over Union (IoU) between this detected bounding box and the original visual instruction box in the input I_{marked}, to rigorously evaluate whether the model adheres to the precise spatial constraints defined by the user's visual markings.

D.3.2 Backward Compatibility Benchmarks

For GEdit-Bench [42] and CEdit-Bench [27], we strictly follow the official evaluation protocols, feeding raw textual editing instructions directly to the models, to verify that our fine-tuning process does not degrade the models' native text-guided editing capabilities. The metrics are defined as:

- **G_SC**: Semantic Consistency, measuring how well the generated result aligns with the input instruction.
- **G_PQ**: Perceptual Quality, evaluating the realism, fidelity, and visual coherence of the generated image.
- **G_O**: The official overall score provided by the benchmarks, combining semantic alignment and perceptual quality.

D.3.3 User Study Full Details

The user study involves 30 volunteers: 10 AI researchers with computer vision background, 10 professional visual designers, and 10 general users without AI expertise, covering both professional and non-professional user groups. We sample 400 test cases in total. For each case, evaluators are shown anonymized outputs from competing methods in a randomized and blinded order, and score each output independently on a 1–5 Likert scale along three dimensions: Spatial Accuracy (alignment with the visual marker intent), Marker Removal (cleanliness of artifact erasure), and Visual Quality (realism and seamless integration of edits). The final score is averaged across all ratings. Randomization is performed independently over both method order and task order to reduce sequence bias.

Ethics Statement. Our user study followed standard human-subject research ethics practices. All participants were informed of the study purpose, evaluation procedure, and data usage before participation, and provided informed consent voluntarily. No personally identifiable information was collected. All responses were anonymized during storage and analysis.

E Additional Experimental Results

E.1 Generalization Evaluation on GEdit-Bench and CEdit-Bench

To verify the generalization capability and robustness of our models in open-domain scenarios, we conducted additional evaluations on GEdit-Bench and CEdit-Bench. While our primary contribution is enabling vision-only editing, it is crucial to verify that this internalization process does not degrade the models' original capability to process standard textual instructions. Therefore, we strictly followed the official evaluation protocols, feeding raw textual editing instructions directly into the models. This setup effectively evaluates whether our models suffer from catastrophic forgetting of text-based control while learning visual markers.

Table 3: Quantitative comparison on GEdit-Bench and CEdit-Bench (Average of EN/CN). Metrics: G_SC (Semantic Consistency), G_PQ (Perceptual Quality), G_O (Overall). All methods use standard **Text Prompts** as input to evaluate backward compatibility.

Method	GEdit-Bench (Avg) ↑			CEdit-Bench (Avg) ↑		
	G_SC	G_PQ	G_O	G_SC	G_PQ	G_O
InstructPix2Pix	3.58	5.49	3.68	3.42	5.31	3.50
Flux-Kontext-dev	7.16	7.37	6.51	6.98	7.22	6.35
AnyEdit	3.18	5.82	3.21	3.05	5.64	3.08
Gemini 2.0	6.73	6.61	6.32	6.55	6.45	6.15
UniWorld-V2	7.56	7.32	7.44	7.40	7.18	7.28
OmniGen2	7.16	6.77	6.41	7.00	6.62	6.25
BAGEL	7.36	6.83	6.52	7.20	6.68	6.36
GPT-Image-1	7.48	7.62	7.42	7.85	7.94	7.70
Step1X-Edit	7.09	6.76	6.70	6.78	7.32	6.30
Qwen-Edit (Base)	7.91	7.82	7.54	8.05	7.81	7.49
LongCat-Edit (Base)	8.13	7.99	7.62	8.26	7.86	7.66
Ours (Siphon) w/ Qwen	**8.17**	**8.11**	**7.84**	**8.33**	**8.15**	**7.80**
Ours (Siphon) w/ LongCat	**8.35**	**8.24**	**7.89**	**8.53**	**8.18**	**7.93**

As shown in Table 3, Siphon preserves the native text-driven editing capability of its respective base models and yields consistent modest gains across both benchmarks. Specifically, Siphon w/ Qwen improves the average Overall Score (G_O) from 7.54 to 7.84 on GEdit-Bench and from 7.49 to 7.80 on CEdit-Bench. Similarly, Siphon w/ LongCat improves G_O from 7.62 to 7.89 on GEdit-Bench and from 7.66 to 7.93 on CEdit-Bench. These results indicate that internalizing visual instructions does not cause catastrophic forgetting of standard text-based control. While the gains are consistent, we interpret this evaluation primarily as evidence of backward compatibility rather than as a claim of statistically significant improvement on general text-guided editing benchmarks.

E.2 Performance Breakdown by Instruction Type

Table 4: CLIP-GT Score breakdown by instruction type. We compare against the strongest baselines to evaluate instruction adherence across modalities.

Method	Type I (Pure Text)	Type II (In-Image Txt)	Type III (Arrow)	Type IV (Box)
InstructPix2Pix	0.28	0.63	0.59	0.60
UniWorld-V2	0.29	0.74	0.65	0.68
OmniGen2	0.31	0.78	0.70	0.72
Qwen-Edit (Base)	0.29	0.76	0.68	0.71
LongCat-Edit (Base)	0.30	0.75	0.67	0.70
Ours (Siphon) w/ Qwen	**0.92**	**0.87**	**0.86**	**0.88**
Ours (Siphon) w/ LongCat	**0.94**	**0.89**	**0.88**	**0.89**

Table 4 details the CLIP-GT scores across different instruction categories, revealing specific strengths of our vision-only approach on both architectures. The most significant gap appears in Type I (Pure Text), where the baselines perform substantially worse (scores $\approx$ 0.30); in contrast, our models achieve $\geq$0.92, proving robust capability to perform implicit OCR and instruction grounding simultaneously. For Type II (In-Image Text), Siphon w/ LongCat scores 0.89, outperforming its base model (0.75). Similarly, in Type III (Arrow) and Type IV (Box), our models achieve scores around 0.86-0.89, demonstrating precise resolution of directional pointers and strict adherence to spatial

Table 5: Spatial ambiguity analysis. Top: separated-text evaluation, where textual instructions are provided separately from visual markers. Prompt-conditioned baselines use their standard text input pipeline, while Siphon is additionally evaluated by feeding the separated instruction through the backbone's original text channel without retraining. We also report Siphon in its default full visual-instruction setting as a reference. MRR is N/A for the text-only setting because the input contains no visual markers to remove. Bottom: hard multi-object disambiguation. CLIP-GT is multiplied by 100. Visual markers provide spatial grounding information that text alone often fails to specify.

Separated-text evaluation				
Setting	CLIP-GT ↑	Target Acc. (%) ↑	Wrong-Obj. Rate (%) ↓	MRR(%) ↑
Base Editor + text only	78.2	72.4	18.9	N/A
Base Editor + marker + text	83.4	79.6	10.8	89.1
VLM Pipeline + marker + text	90.4	89.8	3.6	97.1
Siphon (marker + separated text)	90.9	90.5	3.0	98.2
Siphon (full visual instruction)	**91.3**	**91.2**	**2.7**	**98.4**
Hard multi-object disambiguation				
Method	Target Acc. (%) ↑	Wrong-Obj. Rate (%) ↓	Non-target Pres. (%) ↑	MRR(%) ↑
Qwen-Edit (Base)	63.2	24.7	71.9	89.4
LongCat-Edit (Base)	64.5	23.8	73.1	90.1
OmniGen2	66.7	21.2	75.6	90.3
VLM Pipeline	88.9	4.5	91.3	97.0
Ours (Siphon) w/ Qwen	91.2	3.1	93.6	98.0
Ours (Siphon) w/ LongCat	**92.4**	**2.6**	**94.5**	**98.5**

constraints defined by bounding boxes, ensuring edits are strictly localized and accurate regardless of the underlying backbone.

E.3 Spatial Ambiguity Analysis

We further test whether visual markers provide value beyond being an alternative carrier of text. First, in a separated-text setting, we remove the rendered instruction text from the marked image and provide the editing command through the standard text channel, while keeping only spatial markers such as arrows, boxes, or circles on the image. Second, in a hard-disambiguation setting, each image contains 2–10 similar objects and only one marked target should be edited. We report target accuracy, wrong-object edit rate, and non-target preservation, measuring whether the marked object is correctly edited, whether unmarked similar objects are mistakenly modified, and whether non-target regions remain unchanged.

For the separated-text setting, prompt-conditioned baselines receive the marker-only image and the separated text instruction through their standard input pipeline. We additionally evaluate Siphon under the same marker-plus-separated-text input format: all LoRA adapters are kept frozen, and the separated text instruction is fed through the backbone's original text channel, replacing the fixed placeholder prompt used during fine-tuning. This setting therefore evaluates Siphon under a mild train-inference mismatch, since Siphon is fine-tuned primarily as a vision-only editor and is not exposed to sample-specific text prompts during LoRA training. As a reference, we also report Siphon in its default vision-only setting, where the model consumes the corresponding full visual-instruction image and uses the fixed placeholder prompt as in training.

The separated-text results show that even when the textual instruction is already available through the standard text channel, adding visual markers substantially improves target localization and reduces wrong-object edits. Under the same marker-plus-separated-text setting, Siphon achieves 90.9 CLIP-GT and 90.5% target accuracy, matching and slightly exceeding the VLM pipeline while avoiding an external VLM planner at inference time. Its default full visual-instruction setting performs slightly better (91.3 vs. 90.9 CLIP-GT), which is consistent with our training formulation: Siphon is fine-tuned on the full visual-instruction interface, so feeding text through the backbone's original text channel constitutes a mild train-inference mismatch. The fact that Siphon remains competitive under this diagnostic setting indicates that the gain comes from internalized marker grounding rather than from rendered text alone. The hard-disambiguation benchmark further amplifies this effect: text-driven baselines frequently edit the wrong instance among similar objects, while Siphon reliably localizes the marked target and preserves non-target regions. These results support our central claim that visual markers provide spatial grounding signals complementary to text, and that Siphon internalizes this

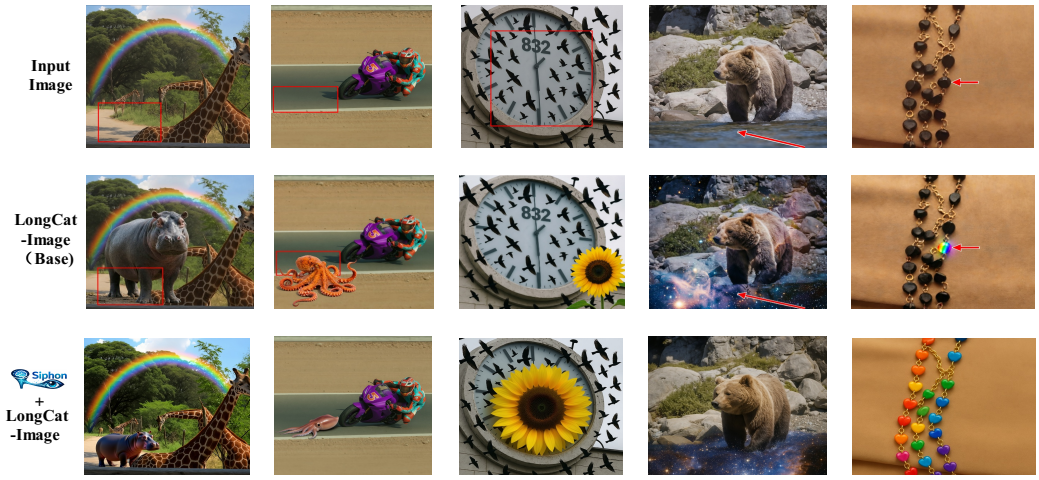


Figure 6: **Qualitative comparison under the separated-text setting.** Each input image contains only spatial markers, such as bounding boxes or arrows, without any embedded instruction text. The editing command is provided separately through the standard text channel using the unified fair prompt template: "Use the visual annotations in the image to perform the following edit: [intent]. Remove all artificial visual annotations from the final image." The bracketed [intent] specifies only *what* to edit, while *where* to edit must be inferred from the visual marker. From left to right, the intents are: "add a hippopotamus," "add an octopus," "add a sunflower," "replace the water with a galaxy-like texture," and "turn the pointed necklace beads into rainbow-colored heart beads." Rows show the marker-only input, LongCat-Edit (Base), and Siphon. The base editor often leaves residual markers, misplaces the edited content, or only partially follows the marked region, whereas Siphon more reliably grounds the edit to the visual marker and removes the artificial annotations.

capability without auxiliary models at inference time. Fig. 6 provides qualitative examples under the same separated-text protocol, where the input image contains only spatial markers and the edit intent is provided through the text channel.

E.4 Additional Ablations: LoRA Rank and Data Synthesis

Ablation A: LoRA Rank. *(Note: For brevity, the LoRA rank and data synthesis ablations are reported on the Qwen-Image-Edit backbone. We observed consistent trends and identical optimal configurations on the LongCat-Image backbone.)* Table 6 indicates that Rank=64 provides the optimal trade-off. Rank=32 suffers from insufficient capacity, evidenced by significantly lower scores in both CLIP-GT (0.81) and CLIP-Text (32.5). Meanwhile, increasing the rank to 128 leads to performance saturation across all metrics, CLIP-GT plateaus at 0.90, and CLIP-Text shows a negligible improvement (+0.1), yet incurs double the parameter overhead (188M vs. 94M).

Table 6: Ablation studies evaluated on Qwen-Edit. **Left:** Impact of LoRA Rank. **Right:** Effect of Data Synthesis Pipeline.

LoRA Rank	Params	CLIP-GT	MRR	CLIP-T
32	47.19M	0.81	92.1	32.5
64 (Ours)	**94.37M**	**0.90**	**97.2**	**34.2**
128	188.75M	0.90	97.5	34.3

Pipeline	CLIP-GT	PSNR	MRR	User
Naive Syn.	0.72	13.9	78.5	3.4
VLM-Guided	**0.90**	**15.7**	**97.2**	**4.3**

Ablation B: VLM-Guided Data Synthesis. Table 6 confirms the necessity of VLM planning. "Naive Synthesis" (raw text) lacks spatial grounding (CLIP-GT 0.72). VLM guidance improves CLIP-GT by 25.0% and MRR by 18.7 percentage points, validating the data engine's effectiveness.

E.5 Additional Ablation on Text-as-Image Forcing

Table 7 reports the detailed TAIF ablation across different backbones and instruction types. Joint Training performs sub-optimally because the model must simultaneously learn visual-text recognition

and spatial grounding. TAIF-Only learns visual-text grounding but does not transfer sufficiently to abstract markers such as arrows and boxes. Sequential TAIF achieves the best overall performance, showing that visual-text grounding provides a useful curriculum before learning marker-based spatial editing.

Table 7: **Ablation on Text-as-Image Forcing (TAIF) across different backbones.** We evaluate the impact of TAIF on different instruction types. *Convergence* denotes steps to reach 90% performance.

Backbone	Method	Type I Acc. (Pure Text)	Type II (In-Image Txt)	Type III (Arrow)	Type IV (Box)	Conv. (Steps)
Qwen-Edit	Joint Training (w/o TAIF)	0.68	0.72	0.61	0.65	∼46k
	TAIF-Only (Zero-Shot)	**0.94**	0.55	0.45	0.41	N/A
	Sequential TAIF (Ours)	0.92	**0.87**	**0.86**	**0.88**	**∼20k**
LongCat-Edit	Joint Training (w/o TAIF)	0.70	0.74	0.63	0.67	∼44k
	TAIF-Only (Zero-Shot)	**0.94**	0.57	0.47	0.43	N/A
	Sequential TAIF (Ours)	0.93	**0.89**	**0.87**	**0.89**	**∼20k**

F Additional Qualitative Results

In this section, we provide extensive qualitative comparisons for each specific instruction type to further demonstrate the robustness of our Siphon framework.

Across all figures in this section (Fig. 7 to Fig. 10), we adopt a consistent six-row layout for a head-to-head comparison: (1) Input images with visual markers, (2) Ground Truth, (3) Qwen-Image-Edit (Base), (4) **Siphon w/ Qwen**, (5) LongCat-Image (Base), and (6) **Siphon w/ LongCat**.

As consistently observed across the distinct instruction types, the base models (Rows 3 and 5) struggle to interpret the visual markers, often treating on-image text, directional arrows, or bounding boxes as meaningless visual noise or textures, thus failing to execute the user's intent. Conversely, integrating our Siphon framework (Rows 4 and 6) empowers both backbones to accurately understand and execute the embedded visual instructions while cleanly erasing the markers.

Specifically, Fig. 7 demonstrates our model's capability in purely text-driven generation via implicit OCR (Type I). Fig. 8 and Fig. 9 showcase the precise semantic execution and referential resolution for in-image text and arrow-guided modifications, respectively. Finally, Fig. 10 highlights the strict spatial adherence achieved in box-guided insertions.

G Generalization to Free-form Hand-drawn Instructions

In real-world applications, users typically interact with editing systems using crude, hand-drawn sketches rather than perfectly rendered geometric shapes. To evaluate the zero-shot robustness of our framework against such unstructured inputs, we test Siphon on images annotated with free-form, hand-drawn markers (e.g., irregular closed loops instead of perfect rectangles, and squiggly, multi-stroke lines instead of straight arrows) accompanied by handwritten-style text. This experiment is intended as a qualitative stress test of out-of-distribution robustness rather than a benchmark-style quantitative evaluation.

As shown in Fig. 11, both base models, **Qwen-Image-Edit** and **LongCat-Image**, struggle to interpret these human-drawn annotations, typically treating the strokes as meaningless visual noise or textures and leaving the image semantically unchanged. In contrast, both **Siphon w/ Qwen** and **Siphon w/ LongCat** generalize well to these out-of-distribution, irregular markers in a zero-shot manner. They accurately ground the spatial constraints of the irregular boxes and resolve the referential targets of the squiggly arrows, executing the intended semantic edits while cleanly erasing the messy strokes. This qualitatively suggests that the internalization process helps the models capture the high-level semantic *intent* of visual pointers, rather than relying purely on low-level pixel patterns.

H Further Discussion and Limitations

Discussion and Analysis: Why Siphon Works? Our results suggest that visual markers reduce the referential ambiguity of text-only control by placing the requested edit near its target. The TAIF

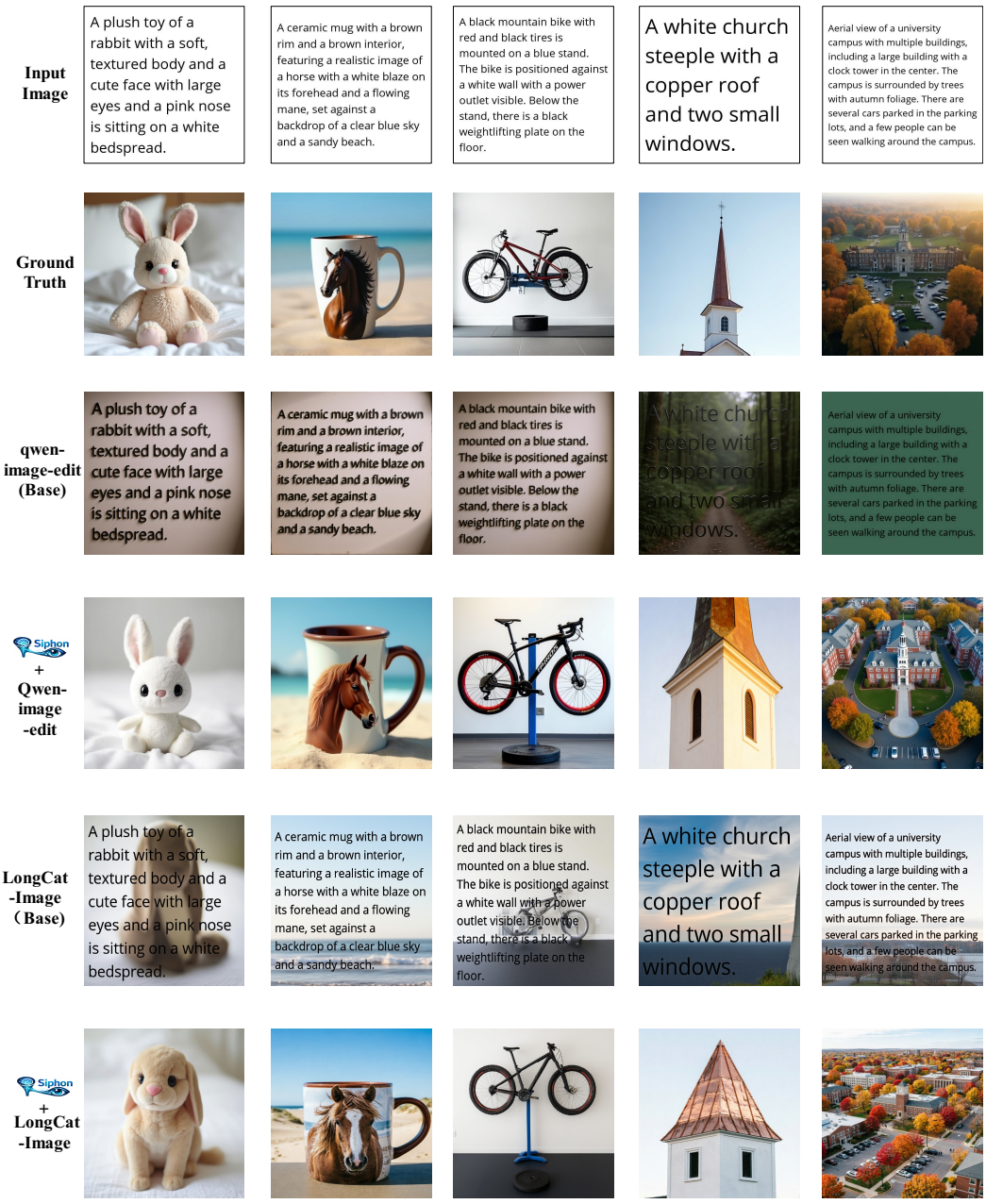


Figure 7: **Additional Results on Type I (Pure Text-to-Image).** Comparison on the text rendering task. The baseline treats the input text image as a texture map, whereas our model successfully interprets the semantic meaning of the glyphs to generate the corresponding object. (Rows top to bottom: Input, Ground Truth, Qwen-Image-Edit Base, Ours w/ Qwen, LongCat-Image Base, Ours w/ LongCat).

Figure 8: **Additional Results on Type II (In-Image Textual Instructions).** Our model effectively detects and executes instructions embedded as text overlays within the scene, while simultaneously removing the text artifacts. (Rows top to bottom: Input, Ground Truth, Qwen-Image-Edit Base, Ours w/ Qwen, LongCat-Image Base, Ours w/ LongCat).

Figure 9: **Additional Results on Type III (Arrow-Guided Modification).** Visualizing the capability to resolve referential ambiguity using arrows. Our model correctly identifies the pointed object and executes the attribute modification. (Rows top to bottom: Input, Ground Truth, Qwen-Image-Edit Base, Ours w/ Qwen, LongCat-Image Base, Ours w/ LongCat).

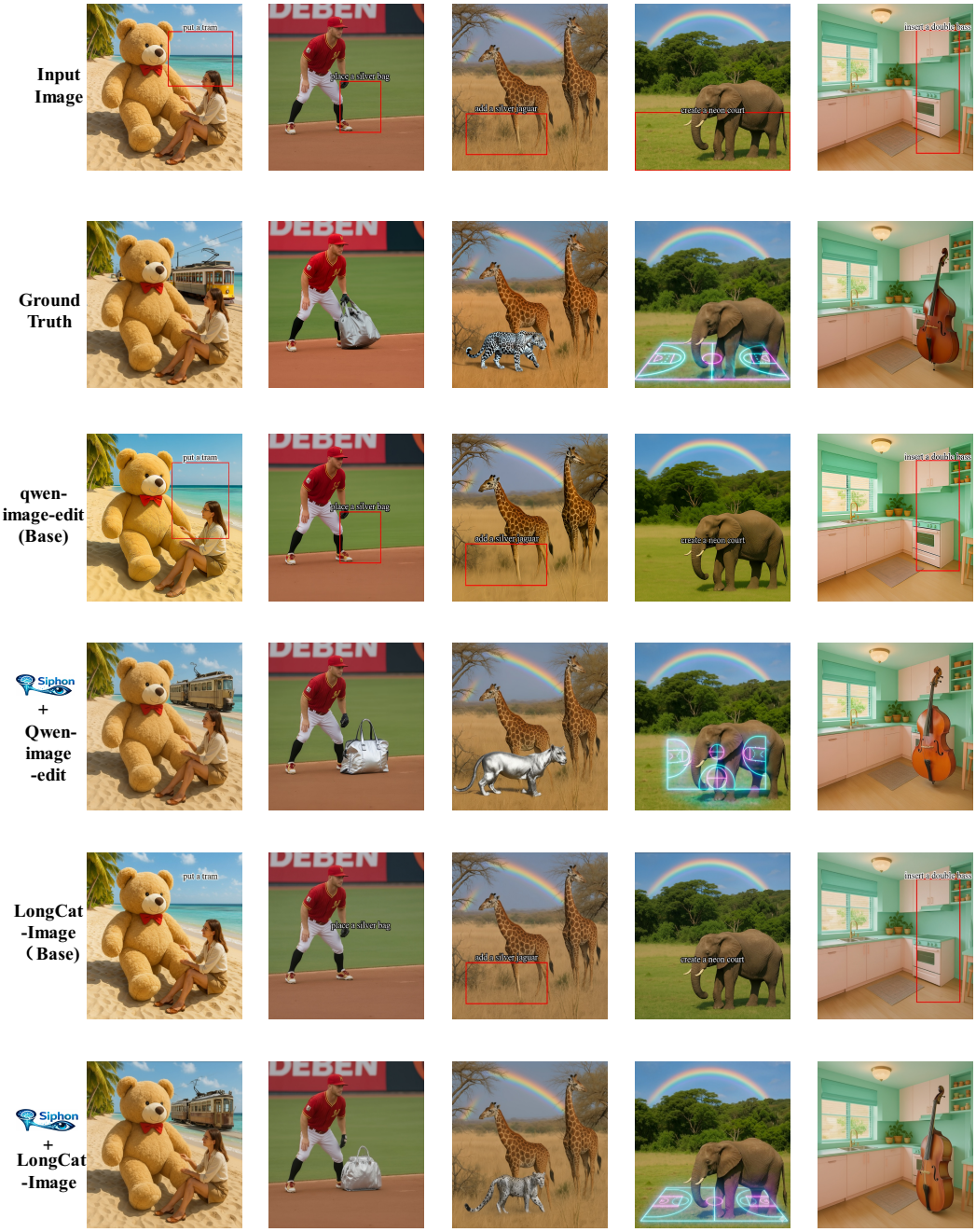
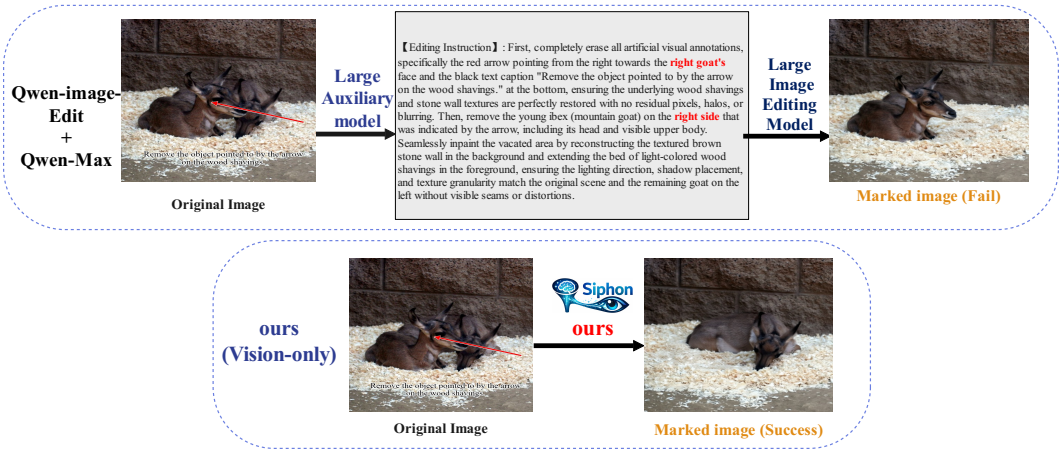


Figure 10: **Additional Results on Type IV (Box-Guided Addition).** Demonstration of precise spatial control. Our model strictly adheres to the bounding box constraints for object insertion and removes the box annotations in the output. (Rows top to bottom: Input, Ground Truth, Qwen-Image-Edit Base, Ours w/ Qwen, LongCat-Image Base, Ours w/ LongCat).

Figure 11: **Zero-shot Generalization to Free-form Hand-drawn Instructions.** We evaluate the robustness of our framework against unstructured, free-form annotations (e.g., irregular boxes, squiggly arrows, and handwritten text). The rows correspond to: (1) Input images with hand-drawn visual markers; (2) Ground Truth; (3) Qwen-Image-Edit (Base); (4) **Siphon w/ Qwen**; (5) LongCat-Image (Base); and (6) **Siphon w/ LongCat**. Observations: Both base models (Rows 3 and 5) fail to comprehend the unstructured annotations, typically treating the crude strokes as meaningless visual noise and leaving the image semantically unchanged. In stark contrast, our Siphon framework applied to both backbones (Rows 4 and 6) successfully interprets the underlying human intent from the irregular sketches, precisely executing the semantic edits and cleanly erasing the hand-drawn artifacts without requiring any additional training on sketch datasets.

ablation supports learning visual-text grounding before mixed editing, while supervision from marked inputs to clean targets encourages joint edit execution and annotation removal. These are empirical interpretations of the training objective and ablations; we do not directly measure an attention-level mechanism.



Generated Prompt by Qwen-VL-Max: "...remove the young ibex (mountain goat) on the **right** side that was indicated by the arrow... Seamlessly inpaint the vacated area..."

Figure 12: **Visualizing Cross-Model Fragility in Pipeline Approaches.** This comparison highlights the risk of cascading errors when relying on an external VLM for spatial reasoning. **Top Row (Pipeline Baseline):** The auxiliary VLM (Qwen-VL-Max) incorrectly resolves the arrow's referential target. Although the visual arrow clearly points to the *left* goat, the VLM suffers from spatial hallucination and erroneously generates a text prompt instructing the editor to remove the *right* goat. The base editor faithfully executes this flawed text prompt, resulting in a completely failed user intent. **Bottom Row (Ours):** By directly internalizing the visual instruction, our end-to-end Siphon framework completely circumvents the fragile VLM-translation step. It accurately resolves the arrow's spatial reference natively from pixels, successfully removing the correct (*left*) goat while seamlessly erasing the visual markers.

Our experiments suggest several takeaways:

1. **Visual markers can offer stronger spatial control than text in our setting.** In our user study, text-driven baselines still lag behind on spatial accuracy and overall preference, which is consistent with the intuition that text-only instructions may suffer from referential ambiguity and spatial over-reach, whereas visual markers provide more explicit spatial cues.
2. **Pipeline approaches can suffer from cross-model fragility.** As illustrated in Fig. 12, reliance on an external VLM for spatial reasoning can introduce failure cases in which an upstream grounding error propagates to the downstream editor. Our end-to-end Siphon framework removes this intermediate translation step at inference time.
3. **Curriculum learning appears important in our training setup.** Decoupling OCR-like alignment from subsequent semantic transfer empirically stabilizes training and improves downstream visual-instruction following.
4. **VLM-guided synthesis provides a practical route for scalable supervision.** The VLM-guided synthetic data engine substantially reduces manual annotation effort and supports large-scale generation of paired training data.

Limitations. Training and evaluation assume that the instruction text and its marker specify the same edit. Conflicting cues (e.g., an arrow pointing to a cat while the text names a dog), ambiguous arrow endpoints, imprecise boxes, or illegible text can cause wrong-target edits or annotation removal without the requested change. Siphon has no conflict detector or clarification mechanism. Free-form

annotation results therefore do not establish robustness to contradictory instructions. External VLMs and teacher editors remain necessary during offline data construction; teacher errors, biases, and coverage constrain the learned capability even though no external planner is required at inference. Broader real-user interactions remain an important direction for future evaluation.

Broader impacts and release considerations. The image-native interface can simplify spatial editing and reduce reliance on external inference services. Improved editing can also facilitate misleading imagery or unauthorized manipulation, and synthetic supervision may inherit teacher biases. We plan to document intended uses, limitations, and applicable asset terms with the release, exclude private or sensitive images, and provide responsible-use guidance. These plans do not constitute a validated misuse-prevention mechanism.

I Additional Evaluation and Data Details

I.1 Instruction-Length Statistics

Table 8 summarizes the rendered instruction lengths in the 4,000-example held-out test set (1,000 examples per type). Type I uses a rendered description and its corresponding natural image; Types II–IV use a marked source and a clean edited target. Construction details appear in Sections 3.1, A, and C.1; the training/test distribution appears in Section D.1.

Table 8: Rendered instruction lengths, measured in words.

Instruction type	Examples	Mean	Median	Range
I: Pure rendered text	1,000	33.7	34	5–50
II: In-image text	1,000	23.6	28	3–50
III: Arrow and text	1,000	7.2	7	3–17
IV: Box and text	1,000	4.5	4	2–12
Overall	4,000	17.3	13	2–50

I.2 Multiple Boxes with Distinct Instructions

We evaluate 300 held-out images, with 100 examples each containing two, three, or four non-overlapping boxes. For identical source images and target edits, instructions are either supplied as a numbered list through the text channel or rendered beside their corresponding boxes. Three independent annotators view the marked input, per-box instructions, and anonymized output. Image-level all-edits accuracy uses majority vote and counts a result as correct only when every requested edit is completed in its corresponding box. The pipeline uses off-the-shelf Qwen-VL-Max and LongCat-Edit without task-specific fine-tuning.

Table 9: Multi-box all-edits accuracy (%). All editors use LongCat.

Method	Format	2 boxes	3 boxes	4 boxes	Average
Base editor	Separated	42.0	25.0	15.0	27.3
VLM pipeline	Separated	71.0	52.0	44.0	55.7
VLM pipeline	Rendered	76.0	59.0	49.0	61.3
Siphon	Separated	74.0	58.0	49.0	60.3
Siphon	Rendered	82.0	69.0	55.0	68.7

Rendering each instruction beside its box improves both the pipeline and Siphon. Under the same rendered input, Siphon exceeds the pipeline while avoiding inference-time VLM planning. Accuracy decreases as the number of requested edits increases.

I.3 External Evaluation on VIBE

We evaluate the complete Addition and Removal subsets of VIBE [54], with 100 examples per task, following its official evaluation protocol. These tasks match our localized insertion and removal

setting. Baseline scores are taken from Table 1 of VIBE; Siphon variants are evaluated under the same protocol. Table 10 reports improvements across both tasks and backbone versions. This evaluation covers these two subsets, rather than all VIBE tasks.

Table 10: VIBE Addition and Removal scores (higher is better).

Method	Addition	Removal
Qwen-Image-Edit	44.20	24.88
Siphon w/ Qwen	58.12	38.65
Qwen-Image-Edit-2509	55.28	14.38
Siphon w/ Qwen-2509	68.45	28.72

I.4 Quantitative Evaluation on Human-Drawn Annotations

We evaluate 300 held-out tasks: 100 each with handwritten-only global instructions, hand-drawn arrows with handwritten instructions, and irregular boxes with handwritten instructions. Ten volunteers annotated with a mouse or stylus without restrictions on handwriting, stroke width, or geometry. Each human-drawn input is paired with a regular rendered annotation sharing the source image, intent, and target. All models are evaluated zero-shot; none of these annotations is used for training. Table 11 uses the main paper's MRR protocol. Both Siphon variants lose 0.05 average CLIP-GT relative to regular annotations while retaining 93–94% MRR.

Table 11: Human-drawn (HD) robustness. CLIP-GT uses the $[0, 1]$ scale; "regular" and "HD avg." average the three modes; arrows and boxes include handwritten instructions.

Method	Regular	Text	Arrow	Box	HD avg.	MRR (%)
Qwen-Edit (Base)	0.71	0.61	0.55	0.58	0.58	71
Siphon w/ Qwen	0.86	0.82	0.79	0.81	0.81	93
LongCat-Edit (Base)	0.70	0.61	0.54	0.58	0.58	72
Siphon w/ LongCat	0.88	0.83	0.82	0.83	0.83	94

Scene text and annotation removal. Training pairs overlay artificial instructions on the source while retaining original scene content in the target. This encourages preserving pre-existing scene text while removing added instructions. Similar-looking scene and instruction text, especially when spatially close, can still be confused.

Supervision quality and existing datasets. Generated pairs undergo VLM-based quality control and manual audits for instruction compliance, spatial consistency, visual quality, and marker removal; low-quality cases are discarded or regenerated.

Inference independence does not remove reliance on offline teachers: their quality and coverage affect supervision. Text-guided pairs from Pico-Banana-400K [38] or OmniEdit [45] could complement our pipeline after constructing visual-instruction correspondences. For compatible object-centric edits, a VLM could identify the subject and operation, Grounding DINO could localize a detectable subject, and text, arrows, or boxes could encode the resulting instruction.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction accurately describe the main contribution of internalizing VLM-assisted visual-instruction-following into a single diffusion editor. The claimed improvements in spatial controllability, instruction adherence, marker removal, and reduced inference-time dependence on external VLM planners are supported by the quantitative results, user study, ablations, and spatial-disambiguation analysis.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper discusses the scope and limitations of the proposed interface, including that long or complex instructions should still be provided through the standard text channel and that on-image text is mainly intended for short labels, visual-text inputs, and spatially grounded edit intents. The appendix also discusses efficiency, practical use cases, and responsible release considerations.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper is primarily empirical and methodological, and does not present new theoretical theorems or formal proof-based results.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper provides the model backbones, LoRA configuration, two-stage training procedure, data distribution, evaluation splits, metrics, baseline protocols, prompt templates, and evaluation details in the main text and appendix. These details, together with the supplemental code and data, are sufficient to reproduce or verify the main experimental claims.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We provide the code, data construction scripts, prompt templates, evaluation scripts, and reproduction instructions in the supplemental material. The supplement includes sufficient details to reproduce the main experiments, including data preparation, training configuration, baseline evaluation, and metric computation.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies the training set size, test split, instruction-type distribution, backbones, LoRA rank and injection layers, optimizer, learning rates, batch size, precision, input resolution, evaluation metrics, and baseline prompt protocol in the main text and appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The user study reports mean and standard deviation and uses paired t-tests to assess statistical significance. For marker removal, the automatic VLM-based MRR metric is calibrated with human annotations, showing 91.5% agreement and Cohen's $\kappa = 0.82$; full repeated training runs for all large diffusion backbones are not reported due to computational cost.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The appendix reports the use of NVIDIA A100 80GB GPUs, the cached-latent training setup, and the inference-time measurements for the base editors and VLM-assisted pipelines. It also provides the training configuration needed to estimate reproduction cost.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: We have reviewed and followed the NeurIPS Code of Ethics. The work uses cited public models and datasets, synthesized visual-instruction supervision, and anonymous human preference ratings without collecting sensitive personal information.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The paper discusses potential positive impacts, such as more intuitive and self-contained visual editing interfaces, as well as potential negative impacts, such as misuse of improved image-editing systems for misleading or manipulated visual content. Responsible-release considerations are discussed in the appendix.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [Yes]

    Justification: The release will include documentation of intended use, limitations, and usage restrictions. Since the method builds on image-editing backbones and may inherit their misuse risks, we will avoid releasing private or sensitive images and will provide responsible-use guidance with the released assets.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper cites the existing models, datasets, and benchmarks used in the work, including the editing backbones, VLMs, public datasets, and evaluation benchmarks. The released package will include attribution and license information for reused assets and will respect their terms of use.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The paper introduces a synthesized visual-instruction dataset, evaluation split, model checkpoints, and a data synthesis pipeline. The appendix documents the instruction types, prompt templates, data distribution, training setup, evaluation protocol, and limitations of these assets.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [Yes]

Justification: The paper reports a blind user study with 30 volunteers who rated randomized outputs on spatial accuracy, marker removal, and visual quality using a 1–5 Likert scale. The study involved anonymous subjective ratings of generated images only, and no sensitive personal information was collected; no monetary compensation was provided.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [No]

Justification: No formal IRB approval was obtained. The user study consisted only of anonymous low-risk subjective ratings of generated images, collected no sensitive personal information, and participants were informed of the study procedure.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The paper explicitly describes the use of a VLM planner, instantiated as Qwen-VL-Max, for visual-instruction synthesis and for the VLM-assisted pipeline baseline. It also describes the use of GPT-4o as an automatic evaluator for marker-removal assessment, calibrated against human annotations.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.